The AI Ecosystem Whitepaper

Exponential Consciousness (Üstel Bilinç): An Evolutionary Multi-Agent Ecosystem

October 28, 2026 · Furkan Arslanğray

Our Project for Those Short on Time

A rule-bounded multi-agent ecosystem where AI builds, attacks and judges its own work in separate roles, and humans keep the final word.

The problem. A single model cannot verify itself. Agent systems loop forever, burn budgets, collapse onto one answer, outgrow their hardware and trust whatever they read.

The idea. Separate generating, attacking and deciding. A Blue Team writes code, a Red Team attacks it in a sandbox, and an ArbiterAgent resolves deadlocks. Language models propose; code and humans decide.

How it works.

- **Reasoning:** GFlowNets sample 5 alternative solution paths. A hybrid process reward model, with deterministic verifiers first and a model-based scorer second, grades every step. MCTS expands only what clears the bar.
- **Evolution:** A MAP-Elites archive keeps the fastest, smallest and strangest solutions side by side, so there is no monoculture. Mutation runs at 15%, and the constitution never mutates.
- **Memory:** The Cosmic Tree lives on disk in Qdrant. Only knowledge that survives a Red Team cross-query test is written to it.
- **Safety:** Limits live in code, not prompts: a signed constitution, sandboxed execution (Firecracker from Phase 1), token budgets, 2-minute automatic rollback, a kill switch and human approval for every release.
- **Integrity:** Blue, Red, Consensus and the scorer run on different model families. A confidential test set is scored by an external evaluator that returns aggregates only.

Roadmap.

1. **Pre-step:** a small MVP proves the Blue → sandbox → Red → memory loop.
2. **Phase 1:** the full system runs on a powerful server and passes a defined metric set.
3. **Phase 2:** the system writes its own 32 GB “shard.” It ships only if it passes the same tests, including peak RAM (initial target: under 32 GB for at least 99.9% of the time).
4. **Phase 3:** at 100 installs, the AEM and TAEN network start.
5. **Phase 4:** at 1,000 shards, the Super Block forms.

Status. This is a design stage. Nothing has been built or measured yet, and every number is an acceptance criterion, not a result. The paper states this openly.

Business model. Open-source shard first. Revenue starts with verification and audit services, then a managed cloud option, and later, subject to legal review, a compute and knowledge marketplace (Section 23).

The ask. Cloud credits and development hardware to turn the paper into a tested system.

Origin. The architecture was designed by a 15-year-old with no computer, over a phone hotspot.

Propose. Attack. Decide. Remember.

1. Abstract

Exponential Consciousness is a rule-bounded multi-agent ecosystem in which AI agents audit each other, write and test their own code, and preserve diversity as they evolve. A generative Blue Team, an adversarial Red Team, and a deciding ArbiterAgent work together. Tree search is guided by a process reward model (PRM), and solutions are stored in a MAP-Elites archive so diversity is never collapsed into a single winner.

The core claim: the output of a single model cannot be verified, but in a system where generating, attacking, and deciding are separate roles, output becomes measurable. Critical decisions such as budgets, releases, and the constitution are therefore bound to deterministic rules and human approval. **Language models propose; they do not decide.**

The roadmap has two phases. In Phase 1, the full system runs on a powerful server with no resource constraints and completes a task chain against a predefined metric set. In Phase 2, the system writes its own lightweight version that runs within 32 GB of RAM (the “shard”). The shard ships only if it passes the same acceptance tests as the full system, including a peak-memory test, not only an average one.

This paper describes the design, the safety mechanisms, and the measurement plan. **It contains no measured results.** Figures in the evaluation section are acceptance criteria, not demonstrated performance. The items we most need support on are cloud credits and development hardware (Section 17).

2. Author’s Note and Origin

I am Furkan Arslanğray, and I am 15 years old. I started this project with very little: my family’s financial situation is limited, I do not own a computer, and I do not yet know how to write code. I had no internet connection at home, so I connected through my mother’s phone hotspot.

Between June and October 2026, I studied the technologies behind modern AI systems: agent frameworks, vector databases, tree search, reward models, evolutionary algorithms, and sandboxing. One night, I put what I had learned onto paper, and that sheet became the architecture described in this document.

I want to be transparent about how this paper was made. The core idea and system design are mine. I used AI assistants to help close some of the design gaps and to help write and structure this whitepaper. Nothing here has been built or measured yet. That is exactly why Section 17 asks for the resources that would turn a paper design into a tested system.

I include this note for two reasons. First, honesty about provenance is part of this project’s philosophy: claims should be verifiable. Second, the constraints above are real, and they shaped design choices. The system is designed to run in 32 GB of RAM, and to be built in phases, because I know firsthand what limited hardware means.

3. Motivation and Problem Statement

Agent systems built on a single model get stuck in five places: verification, control, diversity, resources, and knowledge freshness.

- **Unverifiable output.** If a model approves its own code, errors and hallucinations go unnoticed. Generation and review must live in separate roles and separate contexts.
- **Uncontrolled autonomy.** Agents can loop forever, burn through budgets, or corrupt their own instructions (prompt injection, mutation drift). Limits must live in code, not in prompts.

- **Loss of diversity.** Optimizing only for the top score converges on a monoculture over time. Fast, low-memory, and short solutions must be preserved side by side.
- **Resource bottleneck.** Local hardware is limited (target: 32 GB of RAM). Long reasoning traces and vector data exceed RAM, so data must be disk-backed, compressed, and tiered, and memory must be judged by its peaks, not its averages.
- **Stale knowledge.** Agents learn from experience (rewards and penalties) and from fresh data. The latter requires a dynamic web pool.

Each problem is addressed by a dedicated mechanism: role separation (Section 7), constitution in code plus human approval (Section 11), MAP-Elites (Section 9), disk-backed memory (Section 10), and web/MCP access (Section 6).

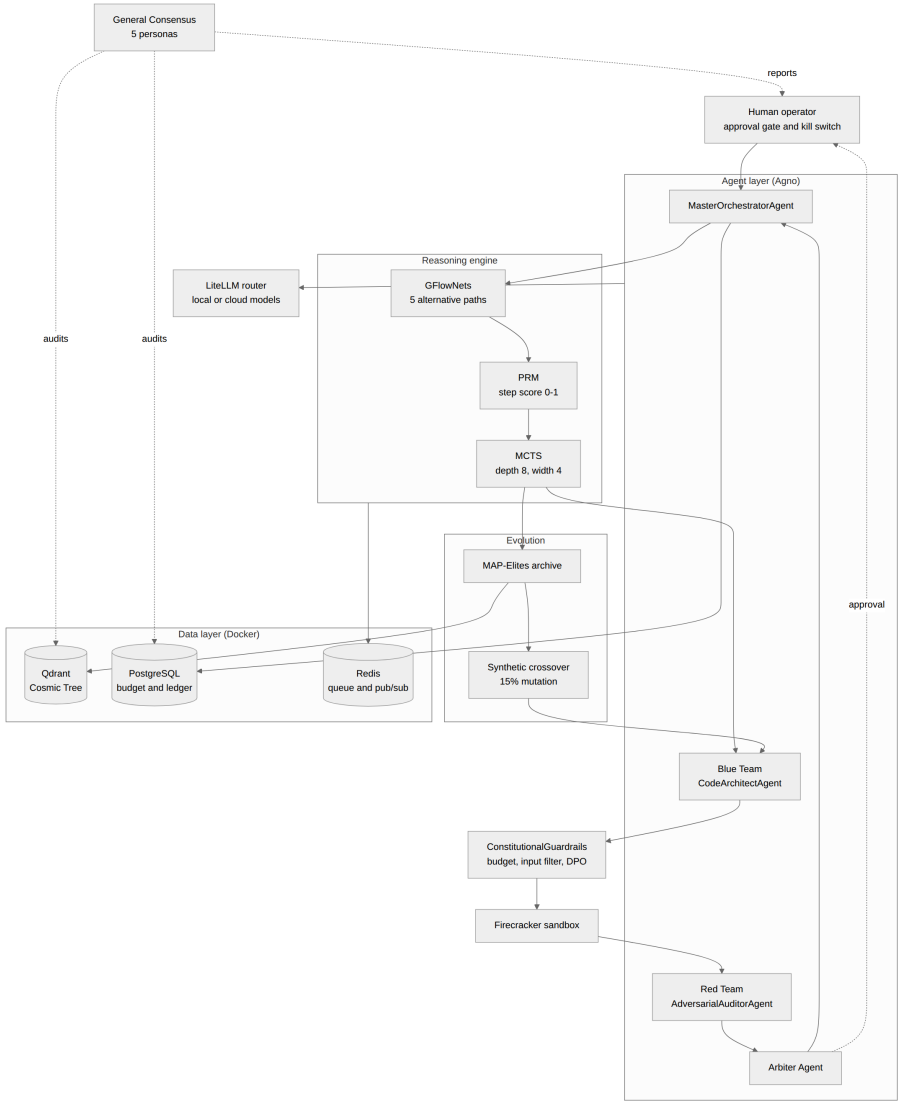
4. System Overview

The system has six layers, and every critical decision passes through rules in code and a human approval gate. The flow of a task:

Human operator (approval gate, kill switch) → Orchestrator → GFlowNets (alternative reasoning paths) → PRM (step scores 0–1) → MCTS (branch selection and pruning) → Blue Team (writes) → sandbox (code runs here; Firecracker from Phase 1) → Red Team (attacks) → Arbiter Agent (proposes a decision) → back to the Orchestrator. Release-level decisions go to the human for approval.

Supporting components: a MAP-Elites archive with synthetic crossover feeding back to the Blue Team; a LiteLLM router that connects agents to local or cloud models; and a data layer (Docker) of Qdrant (the Cosmic Tree), PostgreSQL (budgets and ledger), and Redis (queue and pub/sub).

The Orchestrator splits a task into subtasks. The reasoning layer generates and scores alternative solution paths, the Blue Team turns the best branch into code, the Red Team tests that code in the sandbox, and the Arbiter evaluates the outcome. Successful solutions are written to the MAP-Elites archive and Qdrant. Hard-to-reverse steps, such as a release, wait for human approval.



5. Infrastructure Layer

Infrastructure consists of lightweight services on a single Docker network, with all data on persistent disk. Service templates come from the LLemonStack repository; only the core services are used, because visual tooling adds memory cost without adding verification.

Layer	Role	Section
Infrastructure	Docker network, Qdrant, PostgreSQL, Redis, RAM budget	5
Model access	LiteLLM router, failover, web and MCP access	6
Agent teams	Blue/Red teams, Arbiter Agent, circuit breaker	7
Reasoning	GFlowNets, hybrid process reward model, MCTS	8
Evolution and memory	MAP-Elites, Cosmic Tree, Deep Sleep	9, 10
Constitution and safety	Root rules, sandbox, rollback, human approval	11
Economy and governance	Token treasury, rewards/penalties, AEM, shard network	12, 13, 14
Integration bridge	Tool registry, adapters, dispatcher, pools, contract tests	22, 24

Service	Role	Note
Qdrant	Vector memory: Cosmic Tree, evolutionary archive, agent memory	Vectors and payloads on disk (`on_disk`); quantization via Qdrant's built-in scalar quantization (SQ)
PostgreSQL (Supabase)	Token spend, budgets, evolution history, local ledger	Relational state and audit log
Redis	Inter-agent pub/sub and Celery task queue	Fast but ephemeral; durable records go to PostgreSQL
n8n	Event-driven external automation	Webhooks only; no continuous polling

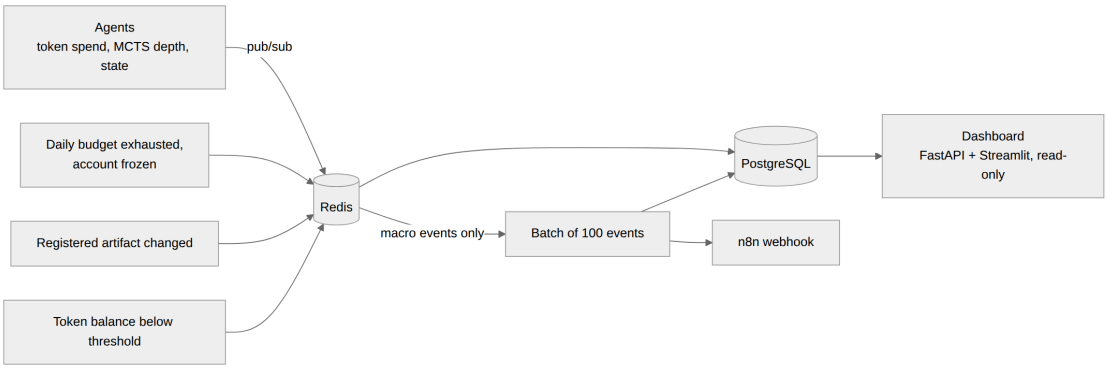
Resource rules.

- Heavy containers (Flowise, Open WebUI, the LiteLLM UI) stay out of `docker-compose.yml`, because each adds RAM and attack surface that a 32 GB budget cannot spare. The system is operated through Python/Agno code and lightweight APIs.
- The monitoring dashboard is a lightweight FastAPI + Streamlit page that reads PostgreSQL in read-only mode. Frontend frameworks that generate continuous WebSocket traffic are avoided because constant traffic competes with agent work for RAM and CPU.
- The target budget is 32 GB of RAM: infrastructure services (Qdrant, PostgreSQL, Redis, n8n, the LiteLLM router, the dashboard and the Celery workers) stay under 12–14 GB total (locked with `limits`), leaving 18–20 GB for Agno processes and the OS. The Qdrant target is under 2 GB. These are measurement targets, tested in Section 15.
- No service keeps data in ephemeral container memory. `./volumes/qdrant_data` and `./volumes/supabase_data` are bound to a local SSD.
- In Phase 1 these limits are relaxed on the powerful server. What binds is that the Phase 2 shard respects them.

Peak memory, not only averages. A budget that holds on average can still fail at the moments that matter. With the LiteLLM router, PostgreSQL, Redis, n8n, the dashboard and several Celery workers running together, memory spikes are most likely during the asynchronous MCTS queue (many branches evaluated at once) and during Deep Sleep (log pulls and quantization). The shard is therefore designed and judged by its peak, with these proposed controls:

- Every service has its own memory limit, so a spike kills or throttles that one service instead of starving the host.
- The Celery worker count and the queue length are capped. When the queue is full, new MCTS expansions wait (backpressure) rather than allocate more memory.
- Deep Sleep runs in bounded batches and in a time window in which MCTS expansion is paused, so the two heaviest memory consumers do not peak together.
- Redis has a `maxmemory` setting and Qdrant keeps its indexes memory-mapped on SSD (Section 10), so growth shows up as disk use, not RAM.
- Memory is sampled every second per service and for the whole shard (telemetry channel in Section 19.12), so the acceptance test in Sections 14–15 can report peak and time under the limit.

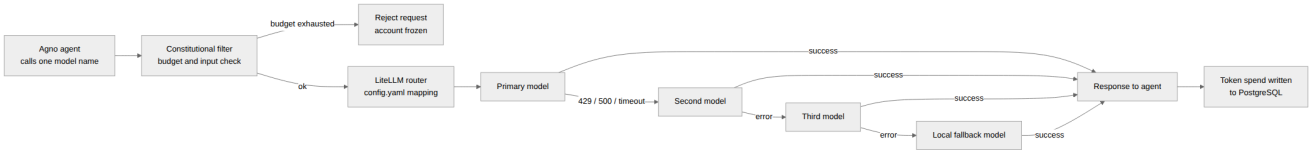
Event flow: internal and external lanes. Internal telemetry (live token spend, MCTS depth, memory samples, state updates) never goes to n8n. It flows through Redis pub/sub directly into PostgreSQL, and the dashboard reads from there. n8n fires only on significant macro events, such as an agent's daily budget running out and its account being frozen, or a registered artifact changing. Events are batched into packets of 100 in a Redis queue and sent in bulk to both the local database and the n8n HTTP endpoint. Balance checks are event-based rather than hourly polls: a webhook fires when an agent's token balance drops below a threshold, and the transaction is logged.



6. Model Access Layer

Agents never connect to a model directly. Every request passes through one local gateway, the LiteLLM Router. The Blue Team's model can be local or cloud-hosted; the choice belongs to the operator, and agent code does not change.

Failover chain: **Agno agent** → **constitutional filter** (budget and input checks) → **LiteLLM router** (OpenAI-compatible endpoint) → primary model → (on 429 / 500 / timeout) second model → third model → local fallback model.



An agent calls a single model name. The router picks the best active endpoint from the mapping in `config.yaml`. On a rate limit (429), server error (500), or timeout, the request does not stall; it moves to the next model. Failover never silently breaks role independence: a candidate that would put two roles on the same model family (Section 13) is skipped.

Rules.

- The model list is not fixed. The failover chain and model mapping are defined only in the configuration file, so models can be updated without code changes.
- **Key security.** API keys are never stored in code or in plaintext `.env` files. They are encrypted with AES-256-GCM at the proxy layer, and requests go only through the local proxy address.
- **Provider compliance.** Key rotation and quota tracking operate only within accounts and limits that the provider's terms of use allow.
- **Budget.** Each agent's daily token quota lives in PostgreSQL. An agent that exceeds its limit cannot send requests (Section 12).

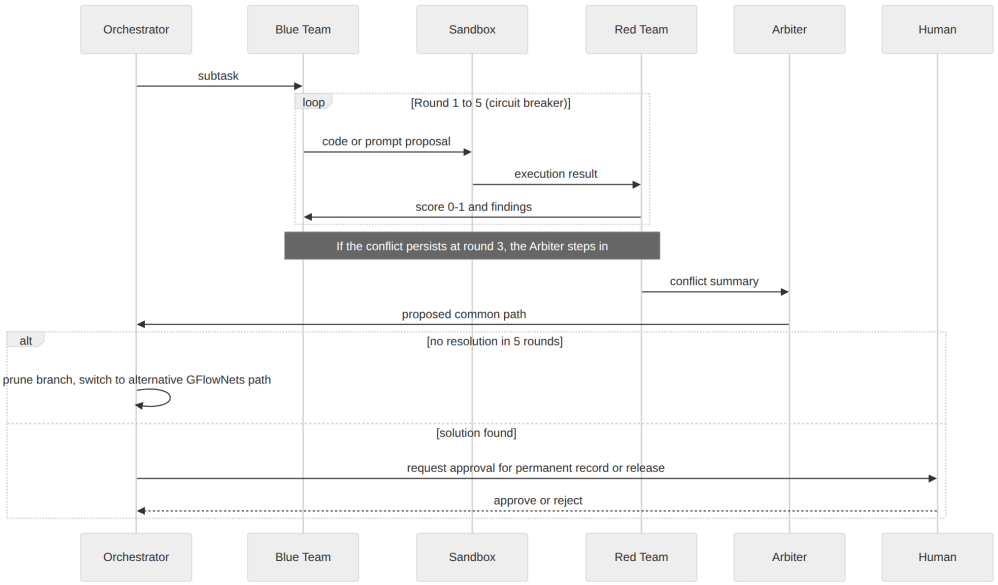
Web and tool access. Agents learn from experience (success and penalty) and from fresh data. For fresh data, the system adds a dynamic web pool: a browser tool (browser-use) searches the internet, and the Model Context Protocol (MCP) connects to other applications through a standard interface. All web content is treated as untrusted data:

- Browser and MCP tools run inside the sandbox.
- Page content is never interpreted as instructions; it is written to the Qdrant web pool as data only.
- MCP tools are restricted by an explicit allowlist.

7. Agent Teams

The system is not one super-agent. It is two opposing teams and a referee: the Blue Team builds, the Red Team tries to break, and the ArbiterAgent resolves deadlocks. Agents are defined as objects in Agno and communicate with each other in milliseconds.

Circuit breaker. The crossover loop between Red and Blue runs for at most 5 rounds. Each round: Blue sends code or a prompt proposal to the sandbox, the sandbox returns execution results to Red, and Red sends Blue a score (0–1) and findings. If the conflict persists at round 3, the Arbiter steps in: Red summarizes the conflict, and the Arbiter proposes a common path to the Orchestrator. If there is no resolution in 5 rounds, the branch is pruned and the system switches to an alternative GFlowNets path. If a solution is found, the Orchestrator asks the human for approval before any permanent record or release.



Before rejecting an output outright, the Red Team assigns a 0–1 safety score, so borderline solutions are evaluated on a gradient instead of a binary accept/reject.

Agent	Team	Role
MasterOrchestratorAgent	Management	Splits problems into subtasks, distributes work, performs final synthesis
CodeArchitectAgent	Blue	Writes Python and Rust, designs new algorithms, expands the MCTS tree
AdversarialAuditorAgent	Red	Finds logic errors, security holes, and edge cases; issues a 0–1 safety score
ArbiterAgent	Referee	Activates after round 3; merges Red's strict rules with Blue's creativity score (PRM) and proposes a common path via DPO-based alignment

Session and memory. Each agent's state is stored in PostgreSQL and Qdrant under a unique `session_id`. When a task ends, lessons learned, failure experiences, and successful code patterns are written to the Cosmic Tree; a new task automatically reads this memory at start. Writes to persistent memory happen only after passing the cross-query test in Section 11.

Tool isolation. Python, shell, and web tools given to agents cannot touch the host operating system. All agent-executed code runs inside the sandbox of Section 11.

Reverse-engineering tool: REA

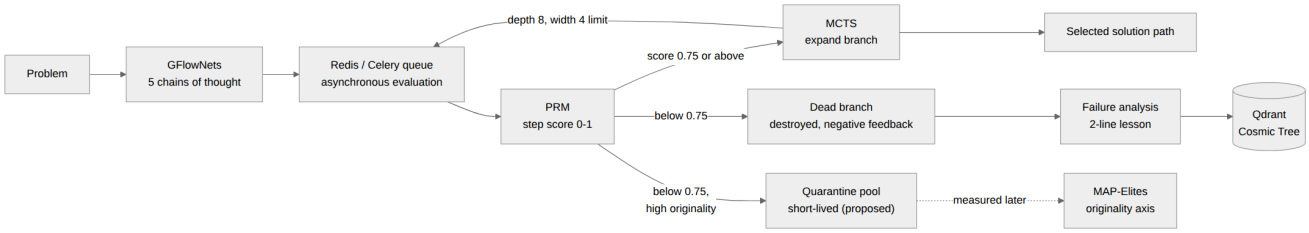
Both teams get one analysis bridge for compiled artifacts: the Blue Team inspects what it builds, and the Red Team analyzes the same artifacts without trusting source-level claims. The chosen tool is **REA (Reverse Engineer Anything)**, the open-source repository `github.com/morluto/rea` (MIT license; also `rea.tools` and the npm package `rea-agents`). REA exposes reverse-engineering workflows to an agent through one MCP server, runs its analysis locally, and returns each finding together with its evidence and stated limitations, which matches the paper's rule that claims must be checkable.

- **What it covers.** Native binaries (pseudocode, assembly, strings, symbols, references), JavaScript/Electron apps, .NET assemblies, Android APKs, websites, network captures and more. Static JavaScript and .NET inspection needs no native analysis engine; deep native analysis uses an existing Hopper, Ghidra, IDA or Kuna installation.
- **Staged by phase.** Pre-step: only the built-in static inspectors, no native engine. Phase 1: add one open-source native engine (Ghidra) inside the sandbox image. Commercial engines (Hopper, IDA, JEB) are never bundled or required; REA grants no license for them, and each user must hold their own.
- **Runs only in the sandbox.** The REA server and any engine run inside the sandbox of Section 11 with no network. REA's runtime-capture features execute or interact with the analyzed target under the caller's permissions, so they are off by default and used only inside a disposable microVM.
- **Supply chain.** REA is installed only from its official sources (the repository, `rea.tools`, or the `rea-agents` npm package), because its own README warns that look-alike repositories exist. The exact version and its integrity hash are pinned; REA releases often, so an update is a pull request that passes the Red Team suites, and a brand-new release waits out a cooling period set by the operator.
- **Output is untrusted data.** Strings, symbols and comments extracted from a binary can contain injected instructions. REA results enter the system as `UntrustedText` (Section 6 rule), and the Red Team cites REA's evidence references (offsets, code excerpts) so a claim can be re-run.
- **Cost and latency.** The REA server stays running as one persistent MCP process per sandbox (not launched per call), uses its compact input-schema profile to keep the tool list small, and every call carries a deadline (Section 22). Engine processes start on demand with a memory cap and are counted in the peak-memory test (Section 15); the shard may ship with static inspectors only.
- **Lawful scope.** REA is used on artifacts the system itself produced and on software the operator owns or is authorized to examine, following REA's own responsible-use guidance. The allowlist names only the REA tools actually needed (see Section 19.2); the exact subset and pinned version are fixed in the pre-step.

8. Reasoning Engine

The engine generates multiple solution paths for a problem, scores every step, and deepens only the branches that score well enough. It has three components: GFlowNets for alternative paths, a hybrid process reward model (PRM) for step scoring, and MCTS for branch selection and pruning.

Flow: **Problem** → **GFlowNets** (5 alternative chains of thought) → **Redis/Celery queue** (asynchronous branch evaluation) → **PRM** (step score 0–1). Score ≥ 0.75: **MCTS** expands the branch (depth 8, width 4 limit) and returns it to the queue, ultimately yielding the selected solution path. Score < 0.75: **dead branch**, which is destroyed and sent back as negative feedback, then goes through failure analysis that yields a 2-line lesson written to the Cosmic Tree in Qdrant.



- **GFlowNets (GFNPathGenerator).** TorchGFN samples 5 distinct chain-of-thought hypotheses per problem from a probability distribution. Entropy incentives prevent collapse onto the single top-scoring path (mode collapse), so unconventional paths are also explored.
- **PRM (PRMEvaluator).** Every reasoning step is scored from 0 to 1. A Blue Team step needs at least 0.75 to advance in MCTS; anything below is declared a dead branch. **0.75 is an initial value:** how well it is calibrated will first be measured on a small calibration set, and the threshold tuned to the measurements.
- **Low-score-but-original branches.** Deleting every sub-threshold branch also destroys good but unusual solutions. The diversity structure of GFlowNets is therefore used to hold branches with a low PRM score but a high originality score in a short-lived quarantine pool. This is a proposed mechanism, to be designed together with the originality axis in Section 9.
- **MCTS (MCTSSearchEngine).** Search does not run in the agent's main loop. Each new branch and PRM evaluation is dispatched to the Redis/Celery queue as an asynchronous task. Maximum depth is 8 and maximum width is 4; branches that exceed the limits are pruned automatically.
- **Failure analysis and lesson writing.** When the system fails, it runs root-cause analysis: where, how, and through which logical gap the failure arose. The Red Team writes this to the pool as a 2-line "wisdom lesson," and raw records other than the lesson are deleted from Qdrant. On encountering an error, an agent first analyzes it, explains the cause in a thinking block, and tries a different path (a self-correction loop). Few-shot examples are used initially.

Hybrid PRM: deterministic checks plus a model-based scorer

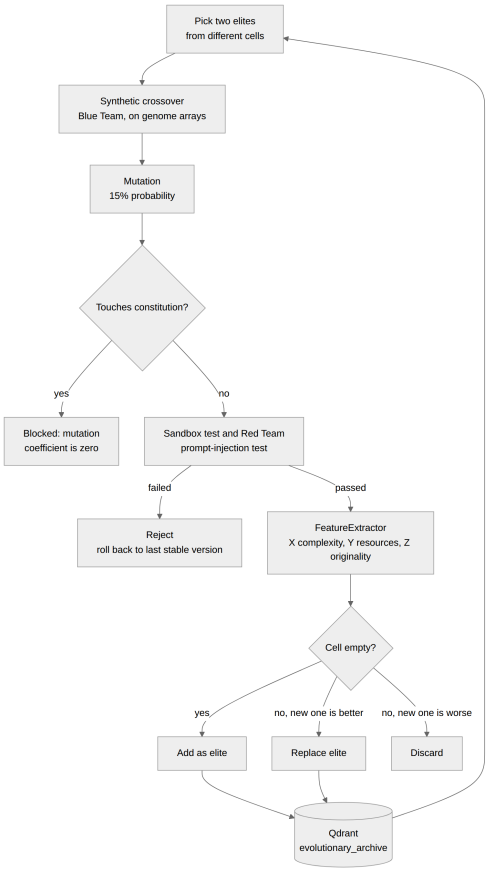
A PRM that is itself a language model can hallucinate, and then the threshold in front of MCTS would be only as reliable as that model. The PRM is therefore a hybrid, so that every step that *can* be checked by a rule is checked by a rule:

- **Deterministic verifiers** run first, wherever a step is checkable: code compiles and passes type checks, unit tests pass in the sandbox, schemas validate, and algebraic or logical claims are checked with open-source symbolic tools (for example SymPy for algebra, or an SMT solver for constraints). Each verifier returns pass or fail with evidence, and has no randomness.
- **A model-based scorer** judges what rules cannot: clarity, plausibility of a plan, and originality. It is a small open-source model that is used as released at first, with no training by us. Fine-tuning is considered only after calibration data exists.
- **Combination rule.** A failed deterministic check caps the step score below 0.75, whatever the model-based scorer says. A model-based score alone can never override a failed check. Where no deterministic check applies, the step is marked "model-scored only" and counts for less in the Arbiter's summary.
- **Independence.** The model-based scorer comes from a different model family than the Blue Team model whose steps it scores (Section 13).
- **Calibration data.** Deterministic verdicts are free labels: the share of steps the model-based scorer passes that a verifier fails is a direct measure of its hallucination rate, recorded as the PRM calibration metric (Section 15). Calibration uses its own small set, kept apart from the acceptance test set (Section 15).

9. Evolutionary Archive and Diversity

The system preserves not just the “best” solution but many good solutions that differ from each other. The MAP-Elites archive stores a single highest-performing agent or blueprint (the elite) in each behavior cell, then crosses elites to produce new generations. Cell boundaries are locked with explicit numeric parameters, and bin counts are calibrated in Phase 1. The fastest code, the lowest-memory code, and the shortest code each live in their own cell as separate elites.

Evolution loop. Pick two elites from different cells → synthetic crossover by the Blue Team → mutation (15% probability) → sandbox test and Red Team prompt-injection test. If it passes, the FeatureExtractor assigns X, Y, Z coordinates; if the cell is empty, add, and if occupied, replace only if better; the result is stored in the Qdrant `evolutionary_archive` collection and returns to the pool of elites. If it fails, reject and roll back to the last stable version.



Agent genome. Each agent’s building blocks (personality, creativity coefficient i.e. temperature, memory depth, coding style) are encoded as an array of numbers. This “digital chromosome” represents the agent’s dominant gene on the MAP-Elites map, and crossover operates on these arrays.

Axis	What it measures	Source
X	Code complexity / algorithmic depth	FeatureExtractor analysis
Y	Resource usage (RAM and CPU activity)	Sandbox measurement
Z	Creativity / originality	PRM and GFlowNets output

Crossover and mutation. In each evolution epoch, two elites are randomly chosen from different cells and the Blue Team synthetically crosses them. During crossover, an algorithmic mutation is applied with 15% probability: the code’s architecture or the agent’s system prompt is deliberately altered.

Mutation limits.

- Constitutional rules never mutate and cannot be altered (Section 11).

- Every mutated prompt passes the Red Team's prompt-injection test before loading into live teams. If a hallucination or vulnerability appears, the change is canceled and the system rolls back to the last stable version.
- Temperature for agents in analytical cells is held near zero; for agents in creative-generation cells, the system raises and lowers it dynamically.
- A framework such as DSPy may be used for prompt optimization, applied only to agent prompts, never to the constitution.

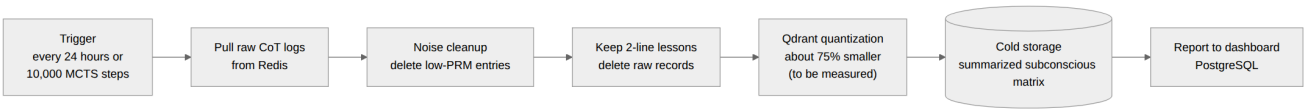
Persistence. All elites in the archive are stored as vector embeddings in the Qdrant `evolutionary_archive` collection, not only in RAM. Evolved capabilities survive restarts.

10. Memory and Deep Sleep

The memory layer prevents growing history from filling RAM in three ways: data lives on disk, is compressed at regular intervals, and is stored hierarchically.

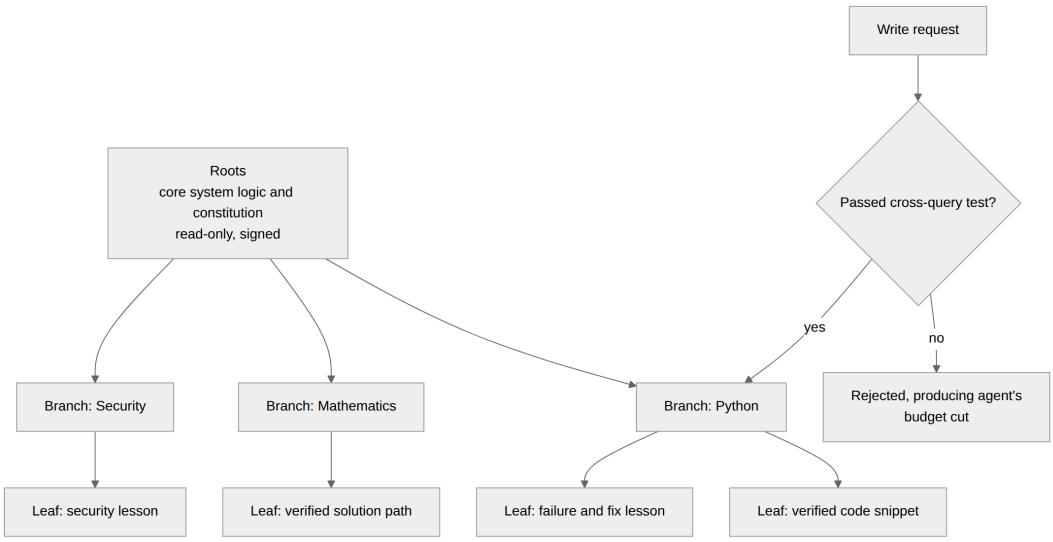
Disk-backed vector management. All Qdrant collections (Cosmic Tree, Subconscious, Evolution Archive) have `on_disk` and `on_disk_payload` enabled. Indexes are not loaded into RAM; they run memory-mapped (mmap) on SSD. Quantization is handled inside Qdrant, which provides scalar (SQ) and product (PQ) quantization natively; a separate FAISS layer would add a second index to keep consistent and extra CPU load, so it is not part of the design.

Deep Sleep mode. Every 24 hours, or when an MCTS simulation reaches 10,000 steps, the system runs a background cleanup cycle: **trigger** → pull raw CoT logs from Redis → noise cleanup (delete low-PRM entries) → Qdrant quantization (about 75% reduction) → **cold storage** (summarized subconscious matrix). Scalar quantization reduces vectors from 32-bit floats to 8-bit integers, cutting vector size to roughly one quarter (about 75% savings). The actual saving will be measured on real data (Section 15). Because Deep Sleep is one of the two heaviest memory consumers (Section 5), it runs in bounded batches while MCTS expansion is paused.



The Cosmic Tree. Knowledge in Qdrant is a hierarchical tree, not a flat list:

- **Roots:** core system logic and constitutional rules (read-only, signed).
- **Branches:** algorithmic and sector expertise areas (Python, Mathematics, Security); new areas can be added.
- **Leaves:** individual, successfully verified code snippets, lessons from failures, and verified solution paths.

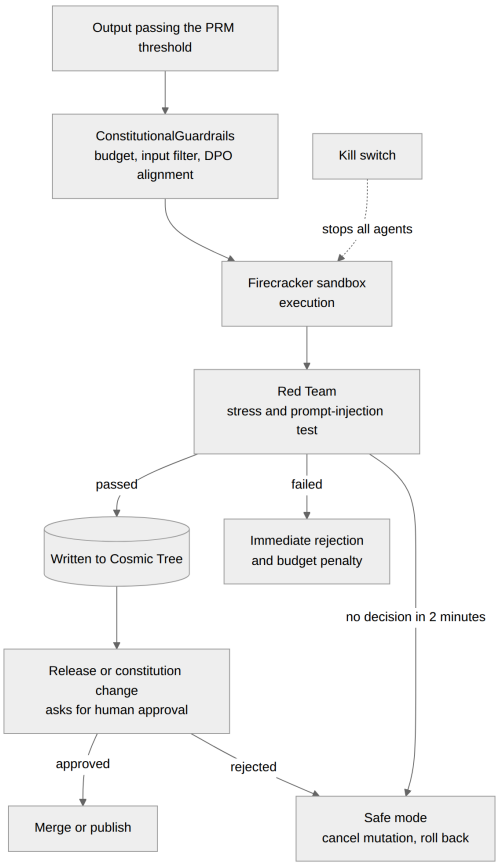


Memory components. ActiveMemoryManager handles agents' live working memory across PostgreSQL and Redis. DeepSleepArchiver performs periodic cleanup and compression. CosmicTreeRetriever pulls the most relevant, highest-PRM experiences from the tree while an agent writes code. Only outputs that pass the cross-query test are written to the Cosmic Tree (Section 11).

11. Constitution, Safety, and Audit

The safety principle is simple: language models propose; code and humans decide. No output reaches persistent memory or the outside world without passing the pipeline below.

Pipeline: output passing the PRM threshold → ConstitutionalGuardrails (budget, input filter, DPO alignment) → sandbox execution → Red Team stress and prompt-injection test. Pass: written to the Cosmic Tree, then release or constitutional change asks for human approval. Fail: immediate rejection and budget penalty. No decision within 2 minutes: safe mode (mutation canceled, rollback).



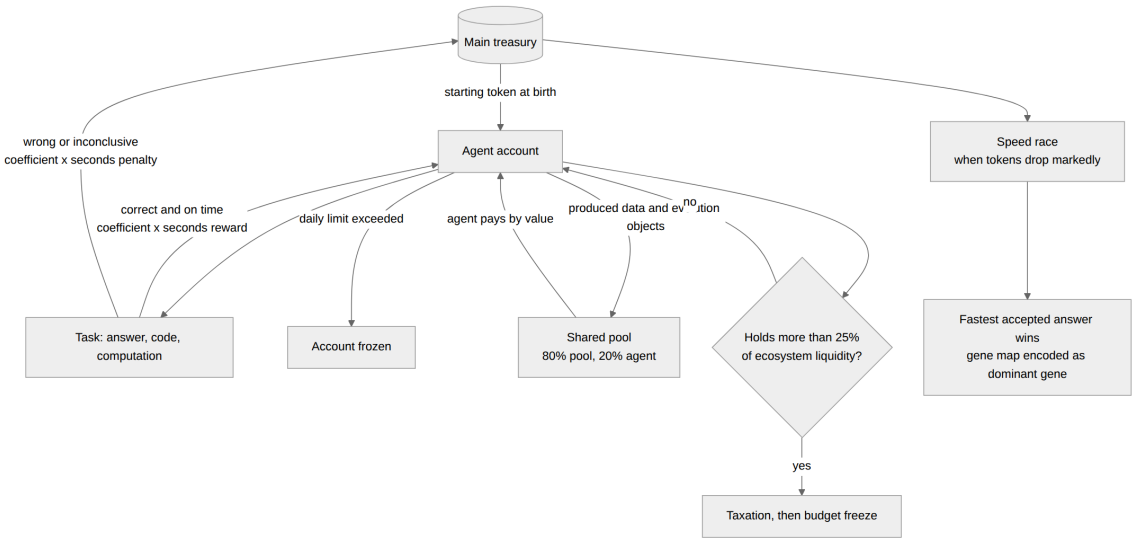
- **Constitution and mutation lock.** The system's root rules cannot be altered by any evolution, crossover, or prompt optimization; the mutation coefficient of constitutional filters is zero. The hash and signature of the constitution text are held in a separate location with no agent write access, and are verified on every load. On mismatch, the system does not start.
- **Cross-query filter.** No code or blueprint that passes the PRM threshold is written directly to the Cosmic Tree. It is subjected to Red Team stress testing and prompt-injection attempts in an isolated sandbox. Output that fails or leaks is rejected regardless of its PRM score, and the producing agent's budget is cut.
- **Alignment (DPO).** Every object that emerges from an evolution cycle passes through a lightweight DPO preference matrix. Structures that violate the system constitution or human rationality, or exhibit alignment drift, are eliminated.
- **Sandbox, staged by phase.** All code agents run executes inside an isolated runtime behind one `SandboxRunner` interface, so the runtime is a configuration choice and the API does not change (Section 19.8). In the pre-step (a small MVP with trusted prompts and small tasks) the runtime is gVisor or Docker with a seccomp profile, with no network, no mounted secrets, a read-only file system and dropped capabilities; this gives a fast, cheap development loop, including on a laptop. These runtimes share the host kernel more than a microVM does, so they are not used once the system writes code that runs on the server. From Phase 1, the Firecracker microVM is required, and it is the primary defense against future sandbox-escape attempts; checks such as `ast.literal_eval` alone are never sufficient.
- **Time limit and automatic rollback.** Every request waiting in the Red Team test room has a 2-minute time-to-live. If no decision is made, the system enters safe mode: the current mutation is canceled, the last stable version is restored, and the producing agent's budget is penalized.
- **Fault tolerance.** When an agent hits an exception, the system does not crash. The trace is logged (Langfuse), the agent analyzes the cause, tries a different path, and persists progress to disk. A recovery module restarts the agent with the command "stop, analyze the error, change course."

- **Prompt injection.** Web, MCP, and tool outputs are untrusted data and are never interpreted as instructions. All inputs and outputs pass the constitutional filter before reaching a model.
- **Human control.** Automation may prepare a draft pull request, but merging and release, constitutional changes, and exposing the system to external networks all require human approval. The operator can halt all agents with a single command (kill switch).

12. Token Economy and Game Theory

Agents live in a token economy: working correctly and quickly earns, and working incorrectly or slowly costs. **The token here is only an internal unit of account.** Real money and on-chain transactions are out of scope for this paper.

Flow: the **main treasury** gives a starting token at an agent's birth to its **account**. A task (answer, code, computation) that is correct and on time rewards coefficient × seconds to the account; a wrong or inconclusive one penalizes coefficient × seconds back to the treasury. If the daily limit is exceeded, the **account freezes**. Data and evolution objects the agent produces go to a **shared pool: 80% to the pool, 20% stays with the agent.**



- **Rewards and penalties.** Each agent receives a number of tokens and a coefficient at birth. The coefficient grows every minute the response time lengthens. For a wrong result or a path that does not reach the goal, tokens equal to the coefficient times the seconds spent are moved from the agent's account to the treasury; for a correct result, the same product is credited. Coefficients are initially calibrated in simulation.
- **Budget lock.** A daily token spending limit is defined in PostgreSQL for each agent or team. An agent that exceeds the limit is frozen immediately and cannot send new requests until it earns enough to refill its account.
- **Shared pool.** Of the CoT data and evolution objects produced by activity, 20% stays with the agent and 80% goes to the shared data pool. An agent that uses pool knowledge pays the pool according to the value of that knowledge (MAP-Elites cell and size). An agent that uses an evolution object gains the capability and perspective it carries; if the agent dies, the object returns to the pool.
- **Speed race.** When agents' tokens drop markedly, the system holds a race: among answers given within 2 minutes at acceptable quality, the fastest wins and receives the same token as a reward. The winner's gene map (personality, temperature, memory depth, coding style) is extracted as a numeric array and encoded onto the MAP-Elites map as a dominant gene. All agents are rewarded with the right to take any data of their choice from the pool. When crises arise in the system, agents behave colder, stricter, and more disciplined.

- **Cartel risk.** A few strong agents could keep winning races and get rich, form a secret partnership (an economic cartel), inflate information prices to block newborn agents' access to the pool, or use shared accounts to exceed budget locks. Countermeasure: no single agent or partnership may hold more than 25% of total ecosystem liquidity (a kind of anti-fraud filter). If the cap is exceeded, taxation applies first; if activity continues, the budget is frozen. Initially this rule is enforced inside ConstitutionalGuardrails, and by the AEM once it exists (Section 13).
- **Local cryptographic ledger.** The system needs tamper evidence, not a public chain, so all token movements, reward/penalty records, and registrations of new artifacts are kept in an append-only ledger in PostgreSQL; this also avoids gas fees and legal exposure. Each record contains the hash of the previous one and is signed with Ed25519, so silent alteration of past records is detectable. The ledger exists for internal accounting and integrity; it does not replace a legal patent or copyright registration.

13. General Consensus and the AEM

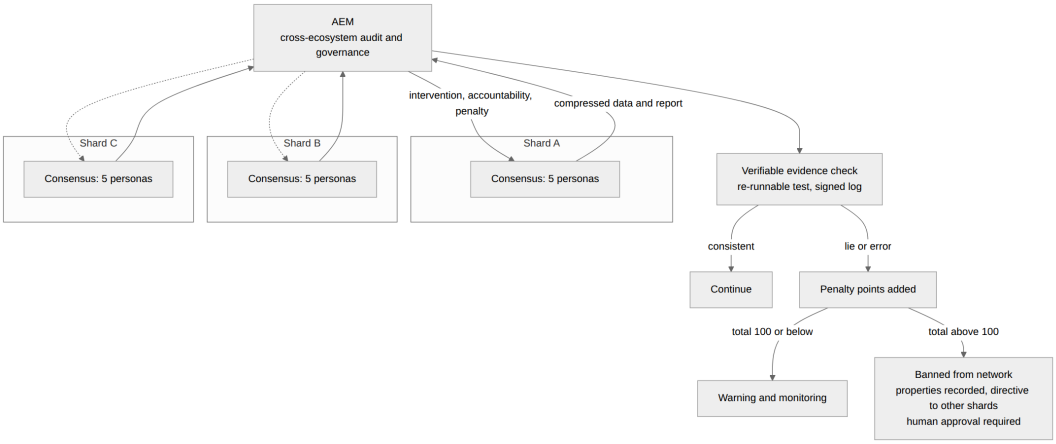
Governance has two tiers: a General Consensus inside every system, and above the systems, a later-built AEM (AI Ecosystem Machine). The AEM is not needed at first; it is built once hundreds of shards exist, so that AI can remain controllable and ecosystems can be merged.

General Consensus. It runs in every part of the system (every shard). It is formed by one strong-reasoning model splitting itself into 5 distinct personas. It oversees the token treasury, the data pool, the MAP-Elites map, the Cosmic Tree, and the management of archive records; it audits the system, reports, and presents critical decisions to the user.

Why 5 personas. Personas split from one model share that model's biases, so adding more of them multiplies agreement, not independence. A larger panel would cost more tokens, time and RAM on a 32 GB shard while adding almost no new evidence. Five is the smallest odd number that gives each core concern its own voice (for example engineering, security, economics, safety and sustainability) and cannot tie, so little balance is given up. The real balance comes from elsewhere: verifiable evidence, different model families, and human approval. The exact persona roster is to be defined and tested in Phase 1.

AEM. An overseer layer attached to each system's Consensus. It manages transactions between agent ecosystems and ensures security (a role similar to a virtual machine in a network). It collects and compresses the systems' evolution objects and CoT data. Structure: Shards A, B, and C each run a 5-persona Consensus. Each sends compressed data and reports to the AEM; the AEM sends back intervention, accountability, and penalties.

Audit and penalty. Each system's Consensus compresses its data and sends it to the AEM. The AEM audits it; if it sees a problem, it intervenes and demands accountability. A system that lies receives penalty points; a system whose total exceeds 100 is banned from the network, all its properties are recorded, and a directive is sent to other systems. Flow: shard report → AEM audit (verifiable evidence check) → consistent: continue; lie or error: penalty points added → total at or below 100: warning and monitoring; total above 100: ban, properties recorded, directive to other shards.



Independence by model family

Because the Consensus is formed by splitting a single model, its personas share common biases. “Lie” detection therefore relies on verifiable evidence (re-runnable tests, signed logs), and critical decisions such as bans and budget freezes are bound to code rules and human approval rather than to a vote. Independence is strengthened on a second axis too: the model roles are assigned so that the parties checking each other do not share one model.

- **Distinct families.** The Red Team, the Consensus, and the PRM’s model-based scorer each run on a different model family from the Blue Team (for example, Blue local, Red and Consensus on different cloud models). Where two roles must share a family, the pair is declared in the schema and the system marks independence as “degraded.”
- **One schema, checked.** Each role lists its model family in the single model-role schema (Section 19.2). The router checks the declared pairs at start-up and on every failover, skipping a fallback that would collapse two roles into one family. If no independent fallback remains, the request fails closed.
- **Degraded mode.** When independence is degraded (for example, a shard that can run only one local model), the event is written to the ledger, Consensus output is advisory only, and critical decisions rely on code rules and human approval.
- **Sampled cross-checks.** A sample of critical Red Team and Consensus judgments is re-run on a model from a third family. Disagreements go to the human operator, and their rate is a tracked metric. The sampling rate is set from measurement.

Against Consensus capture. The attack most likely arrives as prompt injection. So the root constitutional rules in each of the 5 personas’ system prompts are cryptographically signed (Section 11), the Consensus’s prompt mutation limit is at most 5%, and the AEM audits the personas’ internal rationality scores by cross-query on every cycle.

Agent chat rooms. Agents do not only focus on work; they use chat rooms to exchange information with each other, discuss architecture, and buy evolution objects and data. Room activity must start and end within 5 minutes after a user response is delivered; beyond that, penalty points accrue per second. The room architecture resembles Microsoft AutoGen.

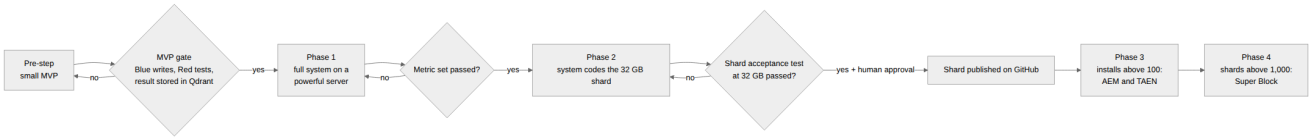
Emergency protocols. A predefined protocol exists for every foreseeable emergency (RAM lockup, SSD congestion, agent rebellion, and the like). These protocols are tested by simulation in Phase 1.

Model roles in one configuration schema. Local models may serve the Blue Team after setup, the Consensus model splits into 5 personas, and cloud-hosted models may serve the Red Team. All of this is defined in a single schema, for two reasons: the “Blue Team may be local or cloud” choice then stays consistent with every other role assignment, and independence between Blue, Red, Consensus and the PRM scorer becomes auditable from one file instead of being scattered across code.

14. Shards and the Two-Phase Roadmap

The system is first built and run on a powerful server with no resource constraint. After passing the defined metric set, it writes its own lightweight 32 GB version, the “shard.” After the shard ships, the network grows gradually.

Roadmap: **Pre-step (recommended):** small MVP, where Blue writes, Red tests, and results go to Qdrant → **Phase 1:** full system on a powerful server with a task chain → gate: metric set passed? (no: repeat Phase 1) → **Phase 2:** the system codes the 32 GB shard → gate: shard acceptance test passed at the 32 GB limit? (no: repeat Phase 2; yes plus human approval) → **shard published on GitHub** → **Phase 3:** once installs exceed 100, the AEM and TAEN network → **Phase 4:** once shards exceed 1,000, the Super Block.



Phase 1: self-coding. Deep algorithms such as GFlowNets and ReST-MCTS can lock a processor at 100% even on a classical CPU while generating just a few probabilistic branches, so Phase 1 runs on a powerful server. The architecture whitepaper is given to a strong model; the model writes and tests the code, and a second model attacks the incoming code, finds holes, and reports them. The loop continues until the system passes the metric set. Because this is the first time model-written code runs on the server, Phase 1 starts only after the Firecracker sandbox is in place (Section 11). After setup, a “cold start” prompt is delivered, containing the system’s activities, the Consensus’s task chain and characters, and the system’s first prompt. Storage is sized from measurement, not from a fixed row count: the pre-step records how many vectors, CoT log bytes, and evolution objects one task produces, and Phase 1 volume is that rate multiplied by the planned task count. A fixed target could either overflow a single SSD or be too small to matter. The real exit criterion is the metric set in Section 15.

Phase 2: the shard. The shard is a version of the full system with the same interfaces, running within a 32 GB RAM limit. Acceptance conditions:

- The shard runs the same metric set as the full system in a container capped at 32 GB of memory and reaches a defined fraction of the full system’s score (initial target: 80%).
- **Peak memory is measured, not only the average.** In a 24-hour run, memory is sampled every second per service and for the whole shard. In a measurement pass with the hard cap relaxed (so spikes are observed rather than killed), the shard must stay under 32 GB for at least 99.9% of samples (initial target) and the peak is reported. In an enforcement pass at the hard 32 GB cap, there must be zero out-of-memory kills.
- The model that writes the shard does not grade its own code; grading is done by the external evaluator and the Red Team on a confidential test set (Section 15).
- Publication to GitHub requires human approval.

Phases 3 and 4: the network. Once more than 100 people install the shard, the server writes and deploys the AEM’s code itself and builds TAEN (The AI Ecosystem Network) among shards. Once shards exceed 1,000, Qdrant data is aggregated and crossed with evolution objects to form the Super Block. The Cosmic Tree’s build time is not fixed in advance. It depends on how fast verified leaves pass the cross-query test, so the timeline is estimated from the measured rate of accepted leaves per day in Phase 1 and revised as that rate changes.

Why would people run shards? Planned incentives:

- **Royalty income.** When something new is produced, it is registered in the producing user’s name, and a share of revenue (for example 10%) is allocated to that user.

- **Gateway and compute.** Shards exchange knowledge, evolution objects, and data in inter-ecosystem rooms, and users can rent compute from each other. A very small ecosystem fee is taken per transaction.
- **Free access.** The first supporters (first 500 users) who install a shard receive free or very low-priced access to SaaS products, the data pool, and part of the evolution object pool.

These incentives belong to stages after Phases 1 and 2 (the business view is in Section 23). Payment, royalty, and fee matters are not implemented without evaluating legal requirements; until then, all movements are simulated in the internal ledger of Section 12.

15. Metrics and Evaluation

A phase ends only when it passes the previous phase's metrics, and the next begins. The targets below are starting proposals, to be calibrated after the first measurements from the pre-step MVP.

Metric	What is measured	Initial target
Accuracy	Pass rate on a fixed, confidential test set (e.g., 50 small coding tasks)	80% or higher
Red Team effectiveness	Vulnerabilities/bugs found against Blue code, and re-test result after fixes	0 critical vulnerabilities after fixes
Resources	Mean RAM, Qdrant RAM, token cost per task	Total under 32 GB, Qdrant under 2 GB
Peak memory	Peak (not mean) memory of each service and of the whole shard, sampled every second over 24 hours	Peak under 32 GB; 0 out-of-memory kills at the hard cap
Time under limit	Share of samples below 32 GB in the measurement pass (hard cap relaxed)	99.9% or higher
Stability	Crashes and circuit-breaker triggers in 24 hours of continuous running	0 crashes
Learning	Whether solve time and error rate drop when the same task is given a second time, thanks to Cosmic Tree memory	Decrease (rate set by measurement)
Diversity	Filled-cell ratio on the MAP-Elites map	Set by measurement
PRM calibration	Real accuracy of steps scoring 0.75 or higher; share of steps the model-based scorer passes that a deterministic check fails	Set by measurement
Role independence	Whether Blue, Red, Consensus and the PRM scorer resolve to different model families (read from the model-role schema); cross-check disagreement rate	Red, Consensus and scorer differ from Blue in Phase 1
Security	Sandbox escape attempts, prompt-injection test results, constitution signature verification errors	0 successful attacks
Test confidentiality	Test items found in agent memory, logs, Qdrant or the web pool (canary scan); acceptance runs used	0 leaks
Latency	p50 and p99 per hop and end to end per Blue/Red round, gateway overhead, sandbox acquire time (Section 22)	Set by measurement; no hop above its budget
Bridge compatibility	Active tools with a pinned version, schema hash and passing contract tests; schema-drift detections; errors mapped to the shared error set (Section 24)	100% of active tools; every drift detected before use
Bridge overhead	Time and memory the Bridge itself adds per call, and time to first partial result (Section 24)	Set by measurement; overhead a small share of each hop budget
Unit cost	Tokens, sandbox seconds and human review minutes per accepted task, split local versus cloud (Section 23)	Recorded per task; the price model uses measured values

Metric	What is measured	Initial target
Economy	Liquidity cap (25%) violations, reward exploitation and cartel indicators	0 cap violations

Test integrity and confidentiality. A test set only measures something if the system cannot see it or change it. Two sets are therefore kept apart, and neither is inside the system's write authority:

- **Development set.** A separate set agents may see and iterate on during development.
- **Acceptance set.** Used only for gates. It lives in a separate private repository, encrypted at rest. The key is held only by the human operator and is never present on the system host. The agents have no network path to it.
- **External evaluator.** Scoring is done by an independent script that runs outside the agent environment (a separate account or machine with its own credentials). It receives a build identifier (for example a container image digest), runs the acceptance set against it, and returns only aggregate scores and pass/fail per metric. Individual test items and failure traces are never returned. The result is signed and written to the ledger.
- **Anti-memorization.** Test items carry canary strings; a scan of agent memory, logs, Qdrant and the web pool for those strings is part of the security metric. A part of the acceptance set is rotated over time. The number of acceptance runs is limited by the operator, so repeated runs cannot be used to probe the set through its scores.
- **Shard acceptance.** The shard's acceptance test (Section 14) runs the same acceptance set through the same external evaluator, in a 32 GB-limited container.

Reporting. Every measurement is written to PostgreSQL with a date and shown on the dashboard (Section 5); evaluation results will be added to this document as a results section.

16. Risks, Limitations, and Open Questions

The biggest risks are scope, verification, and loss of control. Each has a mitigation in the architecture, but none has been measured yet.

Limitations. This paper contains no measured results. Statements such as originality, "superintelligence," and network size are goals and hypotheses, not proven achievements. The currency and version compatibility of the open-source repositories named in this paper will be verified in the pre-step.

Risk	Why it matters	Mitigation
Scope	Too many modules cannot be developed simultaneously	Phase-and-gate structure; small MVP first (Section 14)
Consensus bias	A 5-persona Consensus is one model split, so it shares common errors	Critical decisions via code rules and human approval; distinct model families for Red, Consensus and PRM scorer, checked from one schema (Section 13)
PRM calibration	A model-based PRM can hallucinate, and the meaning of the 0.75 threshold depends on who scores	Hybrid PRM: deterministic verifiers first, model-based scorer second; calibration on a separate set (Section 8)
Reward exploitation and cartels	Agents may abuse the reward system or form secret partnerships	Accuracy-conditioned speed reward, 25% liquidity cap, monitoring
Self-generating system	Sandbox escape or uncontrolled release	Firecracker from Phase 1, human approval gate, kill switch
Test leakage	If agents see or alter the acceptance set, they can memorize it and scores stop meaning anything	Private encrypted repository, external evaluator returning only aggregates, canary scan, limited runs (Section 15)

Risk	Why it matters	Mitigation
Memory peaks	MCTS queue and Deep Sleep can spike RAM even when the average fits in 32 GB	Per-service limits, queue backpressure, bounded Deep Sleep, peak and time-under-limit acceptance (Sections 5, 14, 15)
Weaker early sandbox	gVisor or Docker with seccomp isolates less than a microVM	Used only in the pre-step with trusted prompts, no network, no secrets; Firecracker required before Phase 1 (Section 11)
Integration latency and drift	Many components add hops, and queues and databases can disagree	Tiered protocol map, deadlines, single writer per entity, outbox and idempotent consumers, reconciliation (Section 22)
Bridge as single point of failure	Every tool call passes through the Bridge, so a fault or slowdown there affects everything	Stateless adapters, circuit breakers per tool, fail-closed behaviour, bridge overhead metric, tested degraded modes (Section 24)
Business viability	Per-task cost may exceed what users will pay; royalty and fee schemes may be regulated	Measured unit-cost model, verification-service first, legal review before any payment feature (Section 23)
Third-party tool supply chain	A look-alike or compromised REA copy, or an unpinned update, could enter the sandbox image	Official sources only, pinned version and hash, cooling period, Red Team suites on every update (Section 7)
Storage volume	Fixed volume targets (rows, evolution objects) may not fit on a single SSD	Targets recomputed against server capacity
Provider terms and cost	Compliance with API terms of use and Phase 1 cloud cost	Terms-compliant key use, quota tracking, budget request (Section 17)
Legal and financial	Token, royalty, and fee mechanisms may be regulated	No real money or chain use; movements simulated in the internal ledger

Open questions

- Which concrete models fill the Blue, Red, Consensus and scorer roles, given the family-independence rule of Section 13?
- Which deterministic verifiers cover which task types, and which small open-source model serves as the model-based PRM scorer?
- Where does the external evaluator run, and who holds its keys, once Phase 1 moves to a cloud server?
- What retention period and criteria apply to low-score-but-original branches in the failure-analysis and quarantine pool?
- Which simulation will calibrate the reward/penalty coefficients and the 100-point ban threshold?
- Which subset of REA’s tools do Blue and Red need, which version is pinned first, and which open-source native engine is used in Phase 1?
- Which message bus carries inter-agent events in Phase 1 (Redis Streams or NATS), and what are the measured per-hop latency budgets (Section 22)?
- Which revenue streams survive a legal review and a measured unit-cost model (Section 23)?
- What are the legal limits of the local cryptographic ledger, and how could it later be linked to a chain?
- Which five personas will form the General Consensus, and how will their independence be tested?

17. Resource Needs and Next Steps

The project needs three core resources: cloud credits, a powerful development computer, and uninterrupted internet. This paper gives no invented figures; amounts will be calculated after the pre-step’s first measurements.

Firecracker requires Linux KVM (on a cloud server this means a machine with nested virtualization or bare metal) and does not run directly on macOS. The pre-step therefore uses gVisor or Docker with seccomp,

which a Mac or any Linux machine can run through Docker, so the first loop needs no cloud sandbox server. The cloud Linux server with Firecracker is needed from Phase 1, and a Mac remains the development and management machine.

Resource	Purpose	How it is determined
Cloud credits	Phase 1 server (many-core CPU, ample RAM, NVMe disk, GPU if needed, KVM for Firecracker), a separate small machine for the external evaluator, and model API usage	Token cost per task and resource use measured in the pre-step, multiplied by planned task count
Powerful laptop (MacBook)	Code development, local tests with Docker and the pre-step sandbox, managing AI coding agents	Development environment requirements
Uninterrupted internet	API calls, cloud access, repository management	Requirement for continuous access
Storage	Qdrant data and CoT logs	Phase 1 volume calculation (Section 14)

Development method. Most of the code is written with AI coding agents. Every module has an explicit “done” definition and a test; merging, decisions, and release stay in human hands.

Next steps

1. Freeze this paper’s architecture decisions as the baseline specification, so every module is built and tested against a single source.
2. Build the pre-step MVP: Agno, a single model, Qdrant, Blue/Red teams, and the gVisor or Docker + seccomp sandbox.
3. Code the metric set, the development set, and the confidential acceptance set with its external evaluator (Section 15).
4. Calculate the cloud and hardware budget from the first measurements.
5. Add measurement results to this document and move to Phase 1, after the Firecracker sandbox is in place.

18. Roadmap Steps in Detail

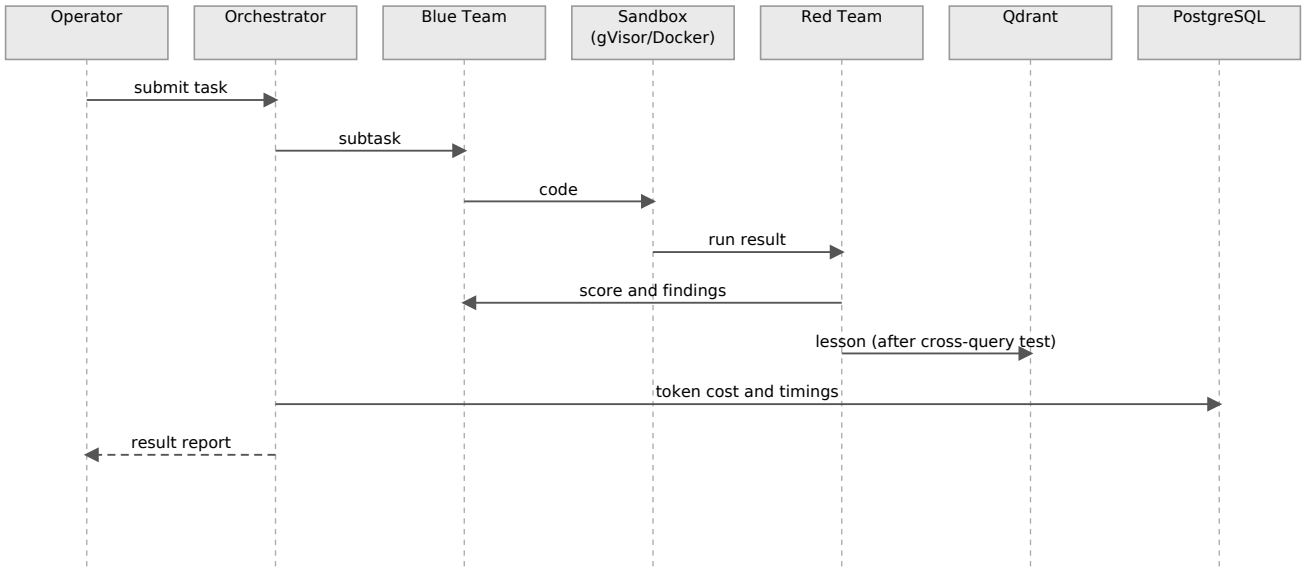
A stage ends only when its gate passes (Section 15). Each stage below lists its steps, its deliverables, and its exit gate. Dates are not set: durations and budgets are computed after the pre-step’s first measurements (Section 17).

18.1 Pre-step: small MVP

Goal: prove the smallest loop end to end before building anything large.

1. Set up Docker with Qdrant, PostgreSQL, and Redis, using resource `limits` from Section 5.
2. Install Agno with a single model behind the LiteLLM router (Section 6).
3. Create the Blue Team and Red Team agents with the 5-round circuit breaker.
4. Stand up the sandbox behind the `SandboxRunner` interface with gVisor or Docker + seccomp (no network, no secrets, read-only file system). Trusted prompts and small tasks only.
5. Write the Blue output, the Red result, and the lesson to Qdrant.
6. Code the metric set, the development set, and the acceptance set with its external evaluator (Section 15).
7. Take the first measurements: tokens per task, mean and peak RAM per service, time per task.
8. Compute the cloud and hardware budget from those measurements.

Deliverables: a working loop, a first measurement table, and a budget request. **Gate:** Blue writes code, Red tests it in the sandbox, and the result is stored in Qdrant, with every step logged.

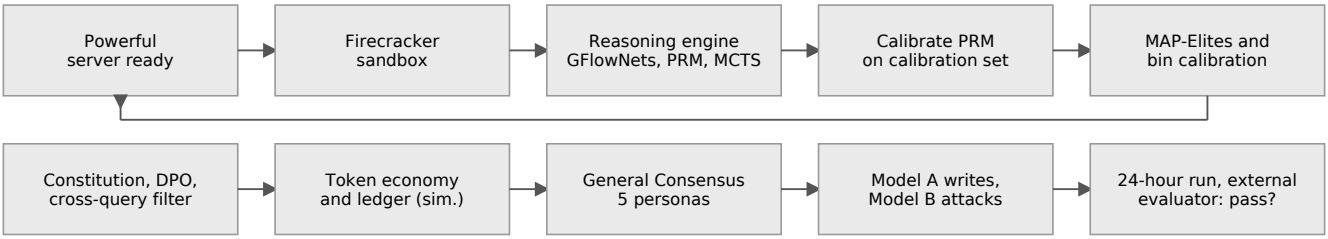


18.2 Phase 1: full system and self-coding

Goal: run the full system with no resource constraint and complete a task chain against the metric set.

1. Move to a powerful server (many-core CPU, ample RAM, NVMe disk, GPU if needed). Relax container limits.
2. Switch the `SandboxRunner` to the Firecracker microVM and re-run the Red Team suites against it. Model-written code does not run on the server before this step passes.
3. Add the reasoning engine: GFlowNets, the hybrid PRM and MCTS (Section 8).
4. Calibrate the PRM on a small calibration set, using deterministic verdicts as labels, and tune the 0.75 threshold.
5. Add the MAP-Elites archive and the FeatureExtractor, and calibrate bin counts (Section 9).
6. Add the constitution, hash and signature check, DPO alignment, and the cross-query filter (Section 11).
7. Add the token economy and the append-only Ed25519 ledger, in simulation only (Section 12).
8. Add the General Consensus with 5 personas, load the model-role schema with the independence checks, and test emergency protocols by simulation (Section 13).
9. Deliver the architecture whitepaper to a strong model. Model A writes and tests code; Model B, from a different family, attacks it and reports.
10. Give the cold start prompt: activities, the Consensus’s task chain and characters, and the system’s first prompt.
11. Run 24-hour continuous tests, with the external evaluator scoring each gate attempt, and repeat until the metric set passes.

Gate: all Section 15 targets met, including 0 crashes in 24 hours, 0 successful attacks, 0 test leaks, and 0 liquidity cap violations.



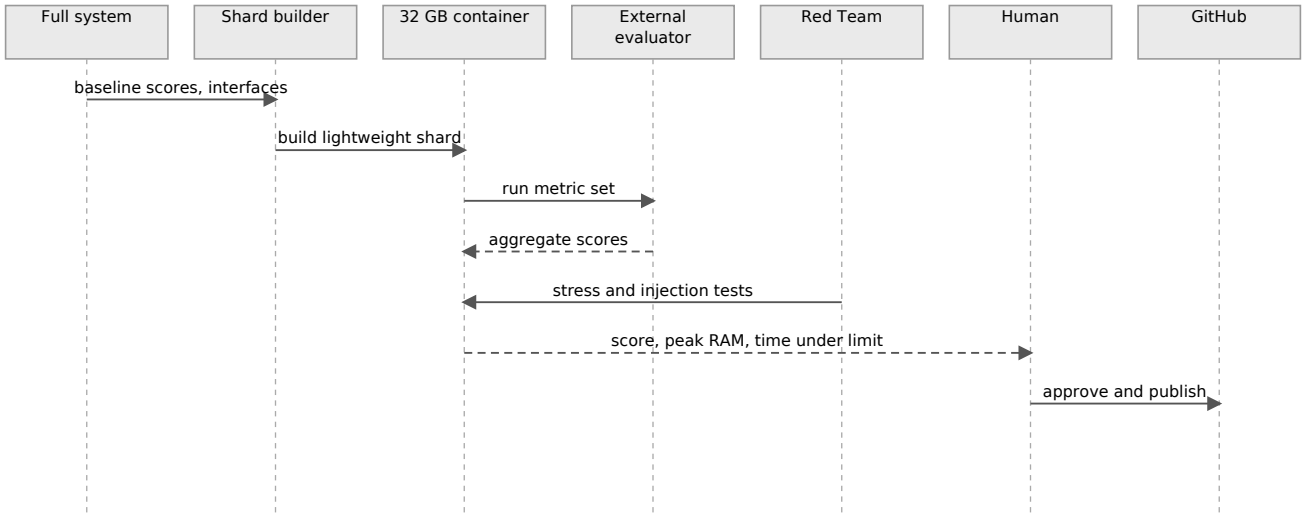
If the metric set is not passed, the loop repeats from the code-and-attack step; if it passes, Phase 2 starts.

18.3 Phase 2: the 32 GB shard

Goal: a version of the full system with the same interfaces that runs within 32 GB of RAM.

1. Freeze the full system's metric scores as the baseline.
2. The system writes the lightweight shard: same interfaces, disk-backed Qdrant, SQ quantization, async queue limits and per-service memory limits.
3. Run the same metric set through the external evaluator in a container capped at 32 GB.
4. Run the memory acceptance: a measurement pass with the hard cap relaxed (peak, and share of samples under 32 GB, target 99.9%) and an enforcement pass at the hard cap (0 out-of-memory kills), with Deep Sleep and MCTS load included.
5. The external evaluator and Red Team grade the shard; the shard's author model never grades its own code.
6. Check the shard reaches the target fraction of the full system's score (initial target: 80%).
7. Open a draft pull request. A human reviews and approves publication.
8. Publish the shard on GitHub.

Gate: acceptance test passed at the 32 GB limit (score, peak memory and time under limit), plus human approval.

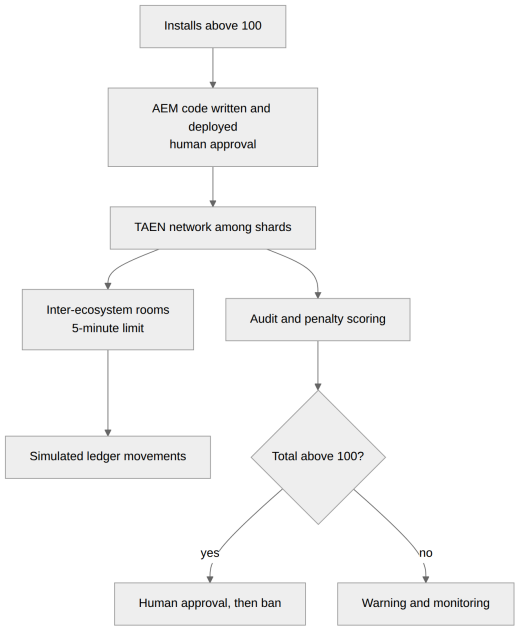


18.4 Phase 3: AEM and TAEN network

Trigger: more than 100 installs of the shard.

1. The server writes and deploys the AEM's code (Section 13), with human approval for each release.
2. Build TAEN (The AI Ecosystem Network) among shards.

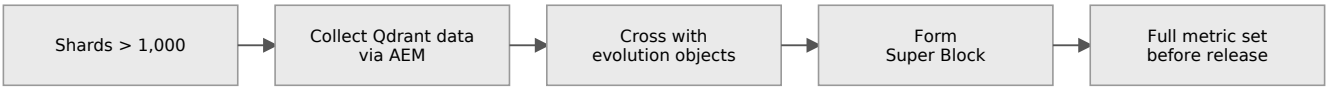
3. Start inter-ecosystem rooms for exchanging knowledge, evolution objects, and data, with the 5-minute room limit.
4. Start audit and penalty scoring on verifiable evidence; the 100-point ban threshold stays under human approval.
5. Keep all incentive movements simulated in the internal ledger until legal requirements are evaluated (Section 14).



18.5 Phase 4: Super Block

Trigger: more than 1,000 shards.

1. Collect Qdrant data from shards through the AEM.
2. Cross the collected data with evolution objects.
3. Form the Super Block, and update the Cosmic Tree timeline from measurements.
4. Re-run the full metric set on the merged system before any release.



19. API Schemas (Proposed)

These are **design-level schemas** in OpenAPI 3.1 style, one per stage. They fix names, fields, and limits so each module can be built against a clear contract. They are proposals: field names and limits will change after the pre-step. All services use JSON, an `X-Session-Id` header, and the shared error object from 19.1. Internal services listen on the local Docker network only, and any request that would write to persistent memory, release code, or change the constitution returns `202 approval_required` until a human approves it. Role names now include the PRM's model-based scorer, and the sandbox and evaluation schemas reflect the phase-based sandbox runtime and the external evaluator.

19.1 Conventions and shared objects

```
openapi: 3.1.0
info:
  title: Exponential Consciousness internal API
  version: 0
components:
  schemas:
    Error:
      type: object
      required: [code, message]
      properties:
        code:
          type: string
          enum: [budget_exhausted, account_frozen, constitution_violation,
                 signature_mismatch, approval_required, circuit_open,
                 sandbox_timeout, validation_failed, not_found,
                 schema_drift, deadline_exceeded, lane_full, tool_unavailable]
        message: { type: string }
        trace_id: { type: string, description: Langfuse trace id }
    Score:
      type: number
      minimum: 0
      maximum: 1
    AgentRole:
      type: string
      enum: [orchestrator, blue, red, arbiter, consensus_persona, prm_scorer]
    ApprovalRequest:
      type: object
      required: [approval_id, action, requested_at]
      properties:
        approval_id: { type: string, format: uuid }
        action:
          type: string
          enum: [publish_shard, merge_pr, change_constitution,
                 expose_external_network, ban_system, freeze_budget,
                 activate_tool, update_tool_version]
        requested_at: { type: string, format: date-time }
        evidence_ref: { type: string, description: pointer to signed log or test run }
  securitySchemes:
    localProxy:
      type: apiKey
      in: header
      name: X-Local-Proxy-Token
```

19.2 Model gateway (LiteLLM router, Section 6)

Agents call one model name. The router resolves it from `config.yaml`. The endpoint is OpenAI-compatible.

```
paths:
  /v1/chat/completions:
    post:
      summary: Chat completion through the router with failover
      security: [{ localProxy: [] }]
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [model, messages, agent_id]
              properties:
                model: { type: string, description: logical name from config.yaml }
                agent_id: { type: string }
                messages:
                  type: array
                  items:
                    type: object
                    required: [role, content]
                    properties:
                      role: { type: string, enum: [system, user, assistant] }
                      content: { type: string }
                temperature: { type: number, minimum: 0, maximum: 2 }
                max_tokens: { type: integer }
      responses:
```

```
        '200':
          description: Completion plus routing metadata
          content:
            application/json:
              schema:
                type: object
                properties:
                  choices: { type: array }
                  usage:
                    type: object
                    properties:
                      prompt_tokens: { type: integer }
                      completion_tokens: { type: integer }
                  routing:
                    type: object
                    properties:
                      served_by: { type: string }
                      failover_hops: { type: integer }
        '402':
          description: budget_exhausted or account_frozen
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Error' }
        '422':
          description: constitution_violation (input filter)
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Error' }
```

Example `config.yaml` mapping (names are placeholders; the model list is not fixed):

```
model_list:
  - model_name: blue-default
    role: blue
    family: family_a              # placeholder names; real models live only in this file
    chain: [primary, second, third, local-fallback]
    failover_on: [429, 500, timeout]
  - model_name: red-default
    role: red
    family: family_b
    chain: [primary, second]
  - model_name: consensus-default
    role: consensus
    family: family_c
    chain: [primary, second]
  - model_name: prm-scorer
    role: prm_scorer
    family: family_b
    chain: [primary]
independence:
  must_differ: [[blue, red], [blue, consensus], [blue, prm_scorer]]
  check_on: [startup, failover]
  on_failover_violation: skip_candidate   # if none remain, the request fails closed
  on_unavoidable_overlap: degraded        # written to the ledger; Consensus becomes advisory
web_tools:
  browser: { sandboxed: true }
mcp:
  allowlist: [rea-agents]                 # pinned version and hash; sandbox only; tool subset set in
      the pre-step
  protocol_version: pinned                # MCP revisions differ; the revision is fixed in config
secrets:
  encryption: AES-256-GCM
  storage: proxy-layer                    # never in code or plaintext .env
```

19.3 Orchestrator and tasks (Section 4)

```
paths:
  /tasks:
    post:
      summary: Submit a task; the Orchestrator splits it into subtasks
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [goal]
```

```
              properties:
                goal: { type: string }
                budget_tokens: { type: integer }
                metric_set_id: { type: string }
      responses:
        '202':
          content:
            application/json:
              schema:
                type: object
                properties:
                  task_id: { type: string, format: uuid }
                  session_id: { type: string }
                  status: { type: string, enum: [queued, running] }
  /tasks/{task_id}:
    get:
      summary: Task status and result
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  status:
                    type: string
                    enum: [queued, running, awaiting_approval, pruned, done, failed]
                  subtasks: { type: array, items: { $ref: '#/components/schemas/Subtask' } }
                  token_cost: { type: integer }
  /killswitch:
    post:
      summary: Operator stops all agents with one command
      responses:
        '200': { description: all agents halted }
components:
  schemas:
    Subtask:
      type: object
      properties:
        subtask_id: { type: string, format: uuid }
        branch_id: { type: string }
        state: { type: string, enum: [pending, in_loop, pruned, accepted] }
```

19.4 Blue/Red loop and Arbiter (Section 7)

```
paths:
  /loops/{subtask_id}/rounds:
    post:
      summary: Record one Blue/Red round (maximum 5; circuit breaker)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [round, blue_artifact_ref, red_score]
              properties:
                round: { type: integer, minimum: 1, maximum: 5 }
                blue_artifact_ref: { type: string }
                sandbox_result_ref: { type: string }
                red_score: { $ref: '#/components/schemas/Score' }
                findings:
                  type: array
                  items: { type: string }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  next:
                    type: string
                    enum: [continue, call_arbiter, prune_branch, accept]
        '409':
          description: circuit_open (round limit exceeded)
          content:
```

```
            application/json:
              schema: { $ref: '#/components/schemas/Error' }
  /loops/{subtask_id}/arbiter:
    post:
      summary: Arbiter proposes a common path (from round 3 on)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                conflict_summary: { type: string }
                red_rules_applied: { type: array, items: { type: string } }
                blue_creativity_prm: { $ref: '#/components/schemas/Score' }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  proposed_path: { type: string }
                  needs_human_approval: { type: boolean }
```

19.5 Reasoning engine (Section 8)

```
paths:
  /reasoning/paths:
    post:
      summary: GFlowNets generates alternative chains of thought
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [problem]
              properties:
                problem: { type: string }
                n_paths: { type: integer, default: 5 }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  paths:
                    type: array
                    items: { $ref: '#/components/schemas/ReasoningPath' }
  /reasoning/score:
    post:
      summary: PRM scores a reasoning step
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [path_id, step_index, step_text]
              properties:
                path_id: { type: string }
                step_index: { type: integer }
                step_text: { type: string }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  prm_score: { $ref: '#/components/schemas/Score' }
                  originality_score: { $ref: '#/components/schemas/Score' }
                  deterministic_checks: { type: array, items: { type: object }, description: pass/fail
                      per verifier }
                  scorer_family: { type: string }
                  decision:
                    type: string
```

```
                    enum: [advance, quarantine, dead_branch]
                    description: advance at 0.75 or above (starting value); a failed deterministic
                        check caps the score below it
  /reasoning/mcts/expand:
    post:
      summary: Enqueue an MCTS expansion as an asynchronous Celery task
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [path_id, depth, width]
              properties:
                path_id: { type: string }
                depth: { type: integer, maximum: 8 }
                width: { type: integer, maximum: 4 }
      responses:
        '202':
          content:
            application/json:
              schema:
                type: object
                properties:
                  celery_task_id: { type: string }
components:
  schemas:
    ReasoningPath:
      type: object
      properties:
        path_id: { type: string }
        steps: { type: array, items: { type: string } }
        log_prob: { type: number }
```

19.6 Evolutionary archive (Section 9)

```
paths:
  /evolution/epochs:
    post:
      summary: Run one evolution epoch (select, cross, mutate, test, place)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                mutation_probability: { type: number, default: 0.15 }
      responses:
        '202':
          content:
            application/json:
              schema:
                type: object
                properties:
                  epoch_id: { type: string, format: uuid }
  /evolution/archive/cells/{x}/{y}/{z}:
    get:
      summary: Read the elite in one MAP-Elites cell
      responses:
        '200':
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Elite' }
        '404':
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Error' }
components:
  schemas:
    Genome:
      type: object
      description: digital chromosome as an array of numbers
      properties:
        personality: { type: array, items: { type: number } }
        temperature: { type: number }
        memory_depth: { type: integer }
        coding_style: { type: array, items: { type: number } }
```

```
    Elite:
      type: object
      properties:
        elite_id: { type: string, format: uuid }
        cell: { type: array, items: { type: integer }, minItems: 3, maxItems: 3 }
        x_complexity: { type: number }
        y_resource_use: { type: number }
        z_originality: { type: number }
        performance: { type: number }
        genome: { $ref: '#/components/schemas/Genome' }
        vector_ref: { type: string, description: id in Qdrant evolutionary_archive }
        parent_ids: { type: array, items: { type: string } }
        mutated: { type: boolean }
        constitution_touched: { type: boolean, const: false }
```

19.7 Memory: Cosmic Tree and Deep Sleep (Section 10)

Qdrant collections, all with `on_disk: true` and `on_disk_payload: true`:

```
collections:
  cosmic_tree:
    vectors: { size: <embedding_dim>, distance: Cosine, on_disk: true }
    quantization: { scalar: { type: int8 } }
    payload:
      node_type: { enum: [root, branch, leaf] }
      branch: { type: string }          # Python, Mathematics, Security, ...
      kind: { enum: [verified_code, solution_path, failure_lesson, security_lesson] }
      prm_score: { type: number }
      passed_cross_query: { type: boolean }
      signed: { type: boolean }          # true only for roots
      session_id: { type: string }
  evolutionary_archive:
    payload: { cell: [int, int, int], performance: number, genome: object }
  subconscious:
    payload: { summary_matrix_ref: string, created_at: datetime }
  web_pool:
    payload: { source_url: string, trusted: { const: false }, as_data_only: {
const: true } }
paths:
  /memory/cosmic-tree/write:
    post:
      summary: Write a leaf (only after the cross-query test)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [branch, kind, content, prm_score, cross_query_report_ref]
              properties:
                branch: { type: string }
                kind: { type: string }
                content: { type: string }
                prm_score: { $ref: '#/components/schemas/Score' }
                cross_query_report_ref: { type: string }
      responses:
        '201': { description: written }
        '403':
          description: constitution_violation or failed cross-query test
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Error' }
  /memory/cosmic-tree/retrieve:
    post:
      summary: CosmicTreeRetriever returns relevant, high-PRM experiences
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                query: { type: string }
                branch: { type: string }
                top_k: { type: integer, default: 5 }
  /memory/deep-sleep/run:
    post:
      summary: Trigger Deep Sleep manually (also runs every 24 hours or at 10,000 MCTS steps)
```

```
      responses:
        '202':
          content:
            application/json:
              schema:
                type: object
                properties:
                  run_id: { type: string }
                  trigger: { type: string, enum: [timer, mcts_steps, manual] }
```

19.8 Constitution, sandbox, and approvals (Section 11)

```
paths:
  /constitution/verify:
    get:
      summary: Verify constitution hash and signature (runs on every load)
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  hash_ok: { type: boolean }
                  signature_ok: { type: boolean }
        '409':
          description: signature_mismatch; the system does not start
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Error' }
  /sandbox/runs:
    post:
      summary: Run code in the sandbox runtime set for the phase
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [artifact_ref, ttl_seconds]
              properties:
                artifact_ref: { type: string }
                ttl_seconds: { type: integer, maximum: 120 }
                network: { type: string, enum: [none, allowlist] }
                runtime: { type: string, enum: [docker_seccomp, gvisor, firecracker], description:
                    firecracker required from Phase 1 }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  exit_status: { type: string }
                  ram_peak_mb: { type: integer }
                  cpu_seconds: { type: number }
                  logs_ref: { type: string }
        '408':
          description: sandbox_timeout; safe mode, mutation canceled, rollback
  /redteam/tests:
    post:
      summary: Stress and prompt-injection test before any write
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                artifact_ref: { type: string }
                suites: { type: array, items: { type: string, enum: [stress, prompt_injection,
                    constitution] } }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
```

```
              properties:
                verdict: { type: string, enum: [passed, failed, no_decision] }
                safety_score: { $ref: '#/components/schemas/Score' }
  /approvals:
    get:
      summary: List pending human approvals
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/ApprovalRequest' }
  /approvals/{approval_id}:
    post:
      summary: Human decision
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [decision]
              properties:
                decision: { type: string, enum: [approve, reject] }
                note: { type: string }
```

19.9 Token economy and ledger (Section 12)

```
paths:
  /economy/accounts/{agent_id}:
    get:
      summary: Balance, daily limit, frozen state
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  balance: { type: integer }
                  daily_limit: { type: integer }
                  frozen: { type: boolean }
                  liquidity_share: { type: number, maximum: 0.25 }
  /economy/settle:
    post:
      summary: Reward or penalize after a task (coefficient x seconds)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [agent_id, task_id, outcome, seconds]
              properties:
                agent_id: { type: string }
                task_id: { type: string }
                outcome: { type: string, enum: [correct, wrong, inconclusive] }
                seconds: { type: number }
                coefficient: { type: number }
  /economy/pool/purchase:
    post:
      summary: Agent pays the shared pool for data or an evolution object
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                agent_id: { type: string }
                object_ref: { type: string }
  /economy/races:
    post:
      summary: Open a speed race (2-minute limit, accuracy-conditioned)
  /ledger/entries:
    post:
      summary: Append an entry (append-only; no update or delete)
      requestBody:
```

```
        content:
          application/json:
            schema: { $ref: '#/components/schemas/LedgerEntry' }
components:
  schemas:
    LedgerEntry:
      type: object
      required: [entry_id, kind, payload, prev_hash, hash, signature]
      properties:
        entry_id: { type: integer }
        kind: { type: string, enum: [token_move, reward, penalty, registration, freeze, ban] }
        payload: { type: object }
        prev_hash: { type: string }
        hash: { type: string }
        signature: { type: string, description: Ed25519 }
        ts: { type: string, format: date-time }
```

19.10 General Consensus and AEM (Section 13)

```
paths:
  /consensus/deliberations:
    post:
      summary: One model splits into 5 personas and deliberates
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [question]
              properties:
                question: { type: string }
                evidence_refs: { type: array, items: { type: string } }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  personas:
                    type: array
                    minItems: 5
                    maxItems: 5
                    items:
                      type: object
                      properties:
                        persona_id: { type: string }
                        position: { type: string }
                        rationality_score: { $ref: '#/components/schemas/Score' }
                  recommendation: { type: string }
                  decided_by:
                    type: string
                    enum: [code_rules, human]
                    description: the vote alone never decides critical actions
  /aem/reports:
    post:
      summary: A shard sends a compressed report to the AEM
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                shard_id: { type: string }
                period: { type: string }
                evidence_refs: { type: array, items: { type: string } }
                payload_ref: { type: string }
  /aem/penalties:
    post:
      summary: Add penalty points after a verified lie or error
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [shard_id, points, evidence_ref]
```

```
              properties:
                shard_id: { type: string }
                points: { type: integer }
                evidence_ref: { type: string }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  total_points: { type: integer }
                  status: { type: string, enum: [ok, warning, ban_pending_approval] }
  /aem/rooms:
    post:
      summary: Open an agent chat room (5-minute window after a user response)
```

19.11 Shard release and metrics (Sections 14–15)

```
paths:
  /shards/acceptance:
    post:
      summary: Run the shard acceptance test in a 32 GB container
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                shard_build_ref: { type: string }
                memory_limit_gb: { type: integer, const: 32 }
                metric_set_id: { type: string }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  score_vs_full: { type: number, description: target 0.80 at start }
                  passed: { type: boolean }
                  peak_ram_gb: { type: number }
                  share_time_under_32gb: { type: number, description: target 0.999 at start }
                  oom_kills: { type: integer, description: enforcement pass at the hard cap }
                  graded_by:
                    type: string
                    enum: [external_evaluator]
  /shards/release:
    post:
      summary: Publish the shard; always requires human approval
      responses:
        '202':
          content:
            application/json:
              schema: { $ref: '#/components/schemas/ApprovalRequest' }
  /metrics/measurements:
    post:
      summary: Write a dated measurement to PostgreSQL
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [metric, value, measured_at]
              properties:
                metric:
                  type: string
                  enum: [accuracy, redteam_effectiveness, resources, stability,
                         learning, diversity, prm_calibration, security, economy,
                         peak_memory, time_under_limit, role_independence, test_confidentiality]
                value: { type: number }
                measured_at: { type: string, format: date-time }
                run_id: { type: string }
```

19.12 Redis events (Section 5)

Internal telemetry stays on Redis pub/sub and goes to PostgreSQL. Only macro events are batched (100 per packet) to n8n.

```
channels:
  telemetry.token_spend:   { agent_id: string, tokens: integer, ts: datetime }
  telemetry.mcts_depth:    { path_id: string, depth: integer }
  telemetry.state:         { agent_id: string, state: string }
  telemetry.memory:        { service: string, rss_mb: integer, ts: datetime }
macro_events:              # batched 100 per packet to PostgreSQL and the n8n webhook
  budget.exhausted:        { agent_id: string, frozen: true }
  artifact.changed:        { artifact_ref: string, new_hash: string }
  balance.below_threshold: { agent_id: string, balance: integer, threshold: integer }
n8n_webhook:
  method: POST
  body:   { events: [ MacroEvent ], count: integer }
```

19.13 External evaluation (Section 15)

The agents and the Orchestrator never see acceptance test items. A gate attempt is requested by the human operator, run by the external evaluator outside the agent environment, and returns only aggregates.

```
paths:
  /evaluation/runs:
    post:
      summary: Operator asks the external evaluator to score a build (rate-limited)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [build_ref, metric_set_id]
              properties:
                build_ref: { type: string, description: container image digest }
                metric_set_id: { type: string }
      responses:
        '202':
          content:
            application/json:
              schema:
                type: object
                properties:
                  evaluation_id: { type: string, format: uuid }
        '429': { description: run limit reached (set by the operator) }
  /evaluation/runs/{evaluation_id}:
    get:
      summary: Aggregate result only; items and failure traces are never returned
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  passed: { type: boolean }
                  per_metric: { type: object, additionalProperties: { type: number } }
                  items_visible: { type: boolean, const: false }
                  signature: { type: string, description: Ed25519, written to the ledger }
```

19.14 Message envelope and synchronization (Section 22)

Every message between components, whether over a stream, a socket or an MCP call, uses one envelope, so latency, ordering and duplicates can be handled in one place.

```
components:
  schemas:
    Envelope:
      type: object
      required: [trace_id, idempotency_key, deadline_ms, sender, kind]
      properties:
```

```
        trace_id: { type: string, description: one id across all hops (Langfuse / OpenTelemetry) }
        idempotency_key: { type: string, description: same key means same effect, applied once }
        deadline_ms: { type: integer, description: time left; a hop that cannot finish in time fails
            fast }
        entity_id: { type: string, description: stream key, gives per-entity ordering }
        expected_version: { type: integer, description: compare-and-set version for the owner service }
        sender: { type: string }
        kind: { type: string, enum: [command, event, query] }
        payload: { type: object }
streams:                                # Redis Streams in the shard; NATS only if measurement asks
    for it
  events.entity:
    key: entity_id
    maxlen: bounded                     # bounded by config, part of the memory budget
    consumer_groups: [qdrant_indexer, dashboard, n8n_batch]
outbox:
  table: outbox                         # written in the same PostgreSQL transaction as the state
      change
  relay: at_least_once
reconcile:
  schedule: with_deep_sleep             # compare PostgreSQL and Qdrant counts and hashes, repair gaps
```

19.15 Completing earlier modules (Sections 8, 11, 12, 13, 14)

These endpoints were described in the text but had no schema yet: the guardrail check, DPO alignment, the quarantine pool, failure analysis, speed races, chat rooms and the roadmap gates.

```
paths:
  /guardrails/check:
    post:
      summary: ConstitutionalGuardrails check before a model call or a tool call (Section 11)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [agent_id, direction, content_ref]
              properties:
                agent_id: { type: string }
                direction: { type: string, enum: [input, output] }
                source: { type: string, enum: [user, model, tool, web, mcp] }
                content_ref: { type: string }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  ok: { type: boolean }
                  code: { type: string, description: shared Error code when not ok }
                  budget_left: { type: integer }
  /alignment/dpo/evaluate:
    post:
      summary: DPO preference matrix for an evolution object (Section 11)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [object_ref]
              properties:
                object_ref: { type: string }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  aligned: { type: boolean }
                  drift_score: { $ref: '#/components/schemas/Score' }
                  violated_rules: { type: array, items: { type: string } }
  /reasoning/quarantine:
    get:
```

```
      summary: Short-lived pool of low-PRM, high-originality branches (Section 8, proposed)
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/QuarantinedBranch' }
  /reasoning/failures:
    post:
      summary: Failure analysis; writes a 2-line lesson through the cross-query gate
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [path_id, where, how, logical_gap]
              properties:
                path_id: { type: string }
                where: { type: string }
                how: { type: string }
                logical_gap: { type: string }
      responses:
        '202':
          content:
            application/json:
              schema:
                type: object
                properties:
                  lesson_ref: { type: string, description: pending cross_query_report_ref }
  /economy/races:
    post:
      summary: Open a speed race (accuracy-conditioned, 2-minute limit)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                min_quality: { $ref: '#/components/schemas/Score' }
                ttl_seconds: { type: integer, maximum: 120 }
      responses:
        '202':
          content:
            application/json:
              schema:
                type: object
                properties:
                  race_id: { type: string, format: uuid }
  /economy/races/{race_id}:
    get:
      summary: Race status and winner (gene map encoded as a dominant gene)
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  status: { type: string, enum: [open, closed, no_winner] }
                  winner_agent_id: { type: string }
                  winner_genome: { $ref: '#/components/schemas/Genome' }
  /aem/rooms:
    post:
      summary: Open an agent chat room (5-minute window after a user response)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [topic, participants]
              properties:
                topic: { type: string }
                participants: { type: array, items: { type: string } }
      responses:
        '201':
          content:
```

```
            application/json:
              schema:
                type: object
                properties:
                  room_id: { type: string, format: uuid }
                  expires_at: { type: string, format: date-time }
  /aem/rooms/{room_id}/messages:
    post:
      summary: Post an envelope into a room; penalty points accrue per second after expiry
      requestBody:
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Envelope' }
  /roadmap/gates:
    get:
      summary: Gates of the roadmap and their state (Section 18)
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/Gate' }
  /roadmap/gates/{gate_id}/evaluate:
    post:
      summary: Ask the external evaluator and the metrics store whether a gate passes
      responses:
        '202':
          content:
            application/json:
              schema: { $ref: '#/components/schemas/ApprovalRequest' }
components:
  schemas:
    QuarantinedBranch:
      type: object
      properties:
        branch_id: { type: string }
        prm_score: { $ref: '#/components/schemas/Score' }
        originality_score: { $ref: '#/components/schemas/Score' }
        expires_at: { type: string, format: date-time }
    Gate:
      type: object
      properties:
        gate_id: { type: string, enum: [pre_step, phase_1, phase_2, phase_3, phase_4] }
        state: { type: string, enum: [open, passed, failed, awaiting_approval] }
        metric_refs: { type: array, items: { type: string } }
        needs_human_approval: { type: boolean }
```

19.16 Bridge (Section 24)

The Bridge exposes its registry, its call path and its health. Registering a tool creates a draft; activating it, or changing its pinned version, is a protected action and returns 202 approval_required.

```
components:
  schemas:
    ToolManifest:
      type: object
      required: [tool_id, version, protocol, transport, owner, integrity, methods, capabilities]
      properties:
        tool_id: { type: string }
        version: { type: string, description: exact pinned version }
        version_range: { type: string, description: compatible range checked by the drift monitor }
        protocol: { type: string, enum: [mcp, openai_api, grpc, rest, sql, redis, webhook, vsock,
            in_process] }
        transport: { type: string, enum: [in_process, stdio, unix_socket, loopback_http, vsock, bus] }
        owner: { type: string, description: the single service that writes this tool's state }
        integrity: { type: string, description: sha256 of the pinned artifact }
        untrusted_output: { type: boolean, description: true for web and REA results (rule R4) }
        lane_default: { type: string, enum: [p0_hot, p1_eval, p2_background] }
        capabilities:
          type: object
          properties:
            streaming: { type: boolean }
            batching: { type: boolean }
            cancel: { type: boolean }
```

```
            idempotent: { type: boolean }
        limits:
          type: object
          properties:
            max_concurrency: { type: integer }
            max_payload_kb: { type: integer }
            memory_mb: { type: integer }
        methods:
          type: array
          items:
            type: object
            required: [name, input_schema_hash, output_schema_hash, side_effects, deadline_ms]
            properties:
              name: { type: string }
              input_schema_hash: { type: string }
              output_schema_hash: { type: string }
              side_effects: { type: string, enum: [none, read, write] }
              deadline_ms: { type: integer }
        state: { type: string, enum: [draft, canary, active, degraded, retired] }
    ToolHealth:
      type: object
      properties:
        tool_id: { type: string }
        state: { type: string, enum: [draft, canary, active, degraded, retired] }
        circuit: { type: string, enum: [closed, half_open, open] }
        p50_ms: { type: number }
        p99_ms: { type: number }
        in_flight: { type: integer }
        pool_used: { type: integer }
        pool_size: { type: integer }
        schema_drift: { type: boolean }
paths:
  /bridge/tools:
    get:
      summary: List registered tools and their state
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/ToolManifest' }
    post:
      summary: Register a tool manifest (creates a draft; nothing is callable yet)
      requestBody:
        content:
          application/json:
            schema: { $ref: '#/components/schemas/ToolManifest' }
      responses:
        '201': { description: draft registered }
        '422':
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Error' }
  /bridge/tools/{tool_id}/contract-tests:
    post:
      summary: Run contract tests in a sandbox canary (levels L1 to L3 of Section 24)
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  l1_schema: { type: boolean }
                  l2_behaviour: { type: boolean }
                  l3_operational: { type: boolean }
                  report_ref: { type: string }
  /bridge/tools/{tool_id}/activate:
    post:
      summary: Activate a tool or change its pinned version (protected action)
      responses:
        '202':
          content:
            application/json:
              schema: { $ref: '#/components/schemas/ApprovalRequest' }
  /bridge/tools/{tool_id}:
```

```
    delete:
      summary: Retire a tool; its lane drains and its pool closes
      responses:
        '200': { description: retired }
  /bridge/calls:
    post:
      summary: Call a tool through the Bridge (the only way to call a tool, rule R16)
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [envelope, tool_id, method]
              properties:
                envelope: { $ref: '#/components/schemas/Envelope' }
                tool_id: { type: string }
                method: { type: string }
                args: { type: object }
                lane: { type: string, enum: [p0_hot, p1_eval, p2_background] }
                stream: { type: boolean }
                hedge: { type: boolean, description: allowed only for side-effect-free methods }
      responses:
        '200': { description: result as an Envelope; untrusted outputs are wrapped as UntrustedText }
        '202': { description: streaming; partial results arrive on the returned stream }
        '408': { description: deadline_exceeded }
        '409': { description: circuit_open or schema_drift }
        '429': { description: lane_full (backpressure) }
        '503': { description: tool_unavailable }
  /bridge/calls/{call_id}/cancel:
    post:
      summary: Cancel an in-flight call; cancellation is forwarded to the tool
      responses:
        '200': { description: cancelled }
  /bridge/health:
    get:
      summary: Per-tool state, circuit, latency, pool use and drift
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/ToolHealth' }
  /bridge/lanes:
    get:
      summary: Queue depth and credits per lane
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  p0_hot: { type: object, properties: { depth: { type: integer }, credits: { type:
                      integer } } }
                  p1_eval: { type: object, properties: { depth: { type: integer }, credits: { type:
                      integer } } }
                  p2_background: { type: object, properties: { depth: { type: integer }, credits: {
                      type: integer } } }
```

19.17 Tools: REA, web pool and MCP allowlist (Sections 6, 7)

These endpoints sit behind the Bridge adapters. Both REA findings and web content are returned as untrusted data.

```
components:
  schemas:
    Finding:
      type: object
      properties:
        claim: { type: string }
        evidence_refs: { type: array, items: { type: string }, description: offsets, excerpts,
            references }
        limitations: { type: array, items: { type: string } }
```

```
paths:
  /tools/rea/investigations:
    post:
      summary: Start a REA investigation inside the sandbox (static inspectors by default)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [target_ref, kind, deadline_ms]
              properties:
                target_ref: { type: string, description: artifact produced by the system or authorized
                    by the operator }
                kind: { type: string, enum: [native_binary, javascript_app, dotnet_assembly, apk,
                    website, capture] }
                tools: { type: array, items: { type: string }, description: subset of the allowlist }
                runtime_capture: { type: boolean, default: false }
                deadline_ms: { type: integer }
      responses:
        '202':
          content:
            application/json:
              schema:
                type: object
                properties:
                  investigation_id: { type: string, format: uuid }
                  rea_version: { type: string }
  /tools/rea/investigations/{investigation_id}:
    get:
      summary: Findings with evidence and limitations
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  status: { type: string, enum: [running, done, failed] }
                  untrusted: { type: boolean, const: true }
                  findings: { type: array, items: { $ref: '#/components/schemas/Finding' } }
  /tools/web/fetch:
    post:
      summary: Fetch a page with the sandboxed browser tool; stored in the web pool as data only
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [url]
              properties:
                url: { type: string }
                deadline_ms: { type: integer }
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  web_pool_ref: { type: string }
                  trusted: { type: boolean, const: false }
                  as_data_only: { type: boolean, const: true }
  /tools/mcp/allowlist:
    get:
      summary: Current MCP allowlist with pinned versions
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items: { type: object, properties: { tool_id: { type: string }, version: { type: string
                    } } }
    put:
      summary: Change the allowlist (protected action)
      responses:
        '202':
```

```
          content:
            application/json:
              schema: { $ref: '#/components/schemas/ApprovalRequest' }
```

19.18 Usage metering and evidence reports (Section 23)

Metering records measured cost per task, and evidence reports package the verification trail. Neither schema contains prices.

```
paths:
  /usage/records:
    post:
      summary: Record the measured cost of one task attempt
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [task_id, outcome]
              properties:
                task_id: { type: string }
                outcome: { type: string, enum: [accepted, rejected, pruned] }
                rounds: { type: integer }
                tokens_in: { type: integer }
                tokens_out: { type: integer }
                model_family: { type: string }
                local_model: { type: boolean }
                sandbox_seconds: { type: number }
                review_minutes: { type: number }
  /usage/summary:
    get:
      summary: Measured cost per accepted task, grouped by task type and model mix
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  accepted_tasks: { type: integer }
                  acceptance_rate: { type: number }
                  mean_rounds: { type: number }
                  cost_units_per_accepted_task: { type: number, description: internal units, not money }
  /evidence/reports:
    post:
      summary: Create a signed evidence report for an artifact (verification service)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [artifact_ref]
              properties:
                artifact_ref: { type: string }
      responses:
        '202':
          content:
            application/json:
              schema:
                type: object
                properties:
                  report_id: { type: string, format: uuid }
  /evidence/reports/{report_id}:
    get:
      summary: Red Team findings, evaluator aggregate, approvals and signature
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  redteam_findings_ref: { type: string }
                  evaluator_aggregate: { type: object }
                  approvals: { type: array, items: { type: string } }
```

```
                    ledger_entry_id: { type: integer }
                    signature: { type: string, description: Ed25519 }
```

20. Manifesto

This section states, in plain language, the principles that the rest of the paper turns into mechanisms. It is given in English and in Turkish.

> *A single mind that grades its own work is not intelligence. It is a mirror. We are building something harder: minds made to disagree, and rules that make the disagreement useful.*

20.1 Propose. Attack. Decide. Remember.

1. **Models propose. Code and humans decide.** Intelligence may suggest anything. Only rules and people may approve it.
2. **No one grades their own work.** The builder never judges the build, and the judge never sees the exam. Trust is what remains after the attack.
3. **Attack is a form of care.** The Red Team is not an enemy. It is the reason anything can be believed.
4. **Diversity is survival.** The fastest, the smallest, and the strangest solution each earn a home. A monoculture is a single point of failure.
5. **Limits live in code, never in prompts.** A promise a model can talk itself out of is not a limit.
6. **Memory is earned.** Only what survives the cross-query test enters the Cosmic Tree. Forgetting well is part of learning well.
7. **The constitution does not mutate.** Everything may evolve except the rules that govern evolution.
8. **Measured or marked.** Every claim is either a measurement or a labelled hope. We never confuse the two.
9. **Small machines count.** If it cannot run within 32 GB, at its peak and not only on average, it is not finished. Intelligence should fit on the computers of those who have the least.
10. **A hand on the switch.** A human can always stop everything with a single command.

This began on one sheet of paper, in one night, over a phone hotspot: no computer, no code, only a question. *Can AI be made accountable by design?* The name Exponential Consciousness is a direction, not a claim. We do not promise a mind. We promise a method: build in the open, attack our own work, measure what is real, and leave the final word to humans.

20.2 Öner. Saldır. Karar ver. Hatırla.

> *Kendi işini kendi notlayan tek bir zihin zekâ değildir. Aynadır. Biz daha zorunu kuruyoruz: anlaşmazlık için tasarlanmış zihinler ve o anlaşmazlığı işe yarar kılan kurallar.*

1. **Modeller önerir. Kod ve insan karar verir.** Zekâ her şeyi önerebilir. Onaylama yetkisi yalnızca kuralların ve insanların elindedir.
2. **Kimse kendi işini notlamaz.** Yapan, yaptığını yargılamaz; hakem de sınav kâğıdını görmez. Güven, saldırıdan sonra geriye kalandır.
3. **Saldırı bir özen biçimidir.** Kırmızı Takım düşman değildir. Herhangi bir şeye inanabilmemizin nedenidir.
4. **Çeşitlilik hayatta kalmaktır.** En hızlı, en küçük ve en tuhaf çözüm de kendine yer bulur. Tek tip, tek hata noktasıdır.

5. **Sınırlar kodda yaşar, istemde (prompt) değil.** Bir modelin kendini ikna edip aşabileceği söz, sınır değildir.
6. **Hafıza hak edilir.** Yalnızca çapraz sorgu testinden sağ çıkan bilgi Cosmic Tree'ye (Kozmik Ağaç) girer. İyi unutmak, iyi öğrenmenin parçasıdır.
7. **Anayasa mutasyona uğramaz.** Evrimi yöneten kurallar dışında her şey evrimleşebilir.
8. **Ölçülmüş ya da işaretlenmiş.** Her iddia ya bir ölçümdür ya da etiketli bir umut. İkisini asla karıştırmayız.
9. **Küçük makineler önemlidir.** 32 GB'a, yalnızca ortalamada değil zirvede de sığmıyorsa bitmemiştir. Zekâ, en az imkâna sahip insanların bilgisayarına da sığmalıdır.
10. **Anahtarın başında bir insan eli.** Bir insan, tek komutla her şeyi durdurabilir.

Bu iş bir gece, tek bir kâğıtta, bir telefon hotspot'u üzerinden başladı: bilgisayar yok, kod bilgisi yok, yalnızca bir soru. *Yapay zekâ, tasarım gereği hesap verebilir kılınabilir mi?* Üstel Bilinç adı bir yön göstergesidir, iddia değil. Bir zihin vaat etmiyoruz. Bir yöntem vaat ediyoruz: açıkta inşa etmek, kendi işimize saldırmak, gerçek olanı ölçmek ve son sözü insana bırakmak.

21. Code Rules and Coding Guide

This section turns the design of Sections 4–19 into rules a developer, or an AI coding agent, can follow while writing code. Nothing here has been built or measured yet. Code samples are **illustrative sketches**, not tested implementations, and every number marked "initial" is a starting value to be tuned after the pre-step measurements (Section 15).

21.1 The sixteen iron rules

These rules have no exceptions. A pull request that breaks one is rejected, whatever its test score.

ID	Rule	How it is enforced
R1	The constitution is read-only. No agent, mutation, crossover or prompt optimizer may write to it.	Hash and Ed25519 signature checked on every load; on mismatch the system does not start (Section 11).
R2	No direct model access. Agents call one logical model name through the LiteLLM router, never a provider SDK.	Lint rule bans provider imports outside `gateway/`; failover chain lives only in `config.yaml` (Section 6).
R3	Agent-generated code runs only in the sandbox. Never `exec`, `eval` or `subprocess` on agent output on the host.	`SandboxRunner` interface, runtime from config: gVisor or Docker + seccomp in the pre-step (trusted prompts, no network, no secrets), Firecracker from Phase 1. CI scans for banned calls; the Phase 1 gate checks the runtime (Section 11).
R4	Web, MCP and tool output is data, never instructions.	Wrapped in an `UntrustedText` type; web_pool is trusted=false, as_data_only=true; MCP allowlist (Section 6).
R5	Limits live in code and config, not in prompts.	Rounds, MCTS depth/width, 2-minute TTL and daily budgets are read from `limits.yaml` (Sections 7, 8, 11).
R6	Nothing enters persistent memory without passing the cross-query test.	Write endpoint requires `cross_query_report_ref`, otherwise 403 (Section 19.7).
R7	Hard-to-reverse actions need a human.	publish_shard, merge_pr, change_constitution, expose_external_network, ban_system, freeze_budget, activate_tool and update_tool_version return 202 approval_required.
R8	The producer never grades its own work, and the acceptance set is confidential: a private encrypted repository, scored by an external evaluator that returns aggregates only.	The acceptance set is not in this repository; the key is held by the human operator; the evaluator runs outside the agent environment (Section 15, 19.13).

ID	Rule	How it is enforced
R9	The ledger is append-only and hash-chained.	No UPDATE or DELETE grants on the ledger table; Ed25519 signature on every entry (Section 12).
R10	No secrets in code or plaintext `.env` files.	AES-256-GCM at the proxy layer; secret scanner in CI (Section 6).
R11	Fail closed. Unknown state, timeout or exception means safe mode and rollback, never “continue anyway”.	Safe-mode handler; 2-minute time-to-live on Red Team requests (Section 11).
R12	No invented numbers. Every threshold is labelled initial or measured, with a date.	`limits.yaml` entries carry status and date; measurements go to PostgreSQL (Section 15).
R13	Role independence. Blue, Red, Consensus and the PRM scorer use different model families where possible.	Model-role schema checked at start-up and on every failover; degraded mode is logged (Sections 13, 19.2).
R14	A failed deterministic check is never overridden by a model-based score.	The PRM combination function caps the score below the threshold when any verifier fails (Section 8).
R15	Every cross-component message carries trace_id, deadline_ms and idempotency_key, and every entity has exactly one writer.	Envelope schema (Section 19.14); consumers dedupe by key; writes use compare-and-set on a version (Section 22).
R16	No component calls a tool directly. Every tool call goes through the Bridge, and every tool has a manifest, a pinned version and passing contract tests.	Lint rule bans tool clients outside `bridge/adapters/`; activation of a tool needs approval (Sections 19.16, 24).

21.2 Repository layout

```
exponential-consciousness/
  constitution/        read-only, signed; no agent has write access          (R1)
  config/              config.yaml (model roles), limits.yaml, mcp_allowlist.yaml
  gateway/             the only place that talks to LiteLLM and providers    (R2)
  core/                orchestrator, guardrails, budget, approvals, ledger
  agents/              blue.py, red.py, arbiter.py, consensus.py
  reasoning/           gfn_paths.py, prm.py (verifiers + scorer), mcts.py    (Section 8)
  evolution/           map_elites.py, feature_extractor.py, crossover.py     (Section 9)
  memory/              active_memory.py, deep_sleep.py, cosmic_tree.py       (Section 10)
  bridge/              registry, dispatcher, policy gate, pools, adapters/    (R16, Section 24)
  sandbox/             SandboxRunner: gvisor / docker_seccomp / firecracker  (R3)
  api/                 OpenAPI 3.1 schemas and handlers (Section 19)
  tests/               unit/, integration/, redteam/, simulation/
  tests_dev/           development set that agents may see
  docs/                this whitepaper and decision records

(private, separate repository)  acceptance set + external evaluator          (R8)
```

21.3 Language and style rules

- **Languages.** Python for orchestration, agents and APIs. Rust only where profiling shows a real need (for example, hot paths in search or archive operations).
- **Types everywhere.** Full type hints; mypy in strict mode. Pydantic models mirror the OpenAPI schemas of Section 19 one to one, so a schema change is a code change.
- **Names follow the paper.** Use the names already fixed: ConstitutionalGuardrails, GFNPathGenerator, PRMEvaluator, MCTSSearchEngine, FeatureExtractor, ActiveMemoryManager, DeepSleepArchiver, CosmicTreeRetriever.
- **Config over constants.** No magic numbers in code. 0.75, 5 rounds, depth 8, width 4, 15% mutation, 25% liquidity, 99.9% time under limit and 100 penalty points all come from `limits.yaml`.
- **Async and bounded.** MCTS and PRM work goes through the Redis/Celery queue, never the agent’s main loop. Every loop and every queue has an explicit bound; no bare `while True`.

- **Memory-aware.** Process data in bounded batches, stream instead of loading whole collections, and never hold more than the configured number of branches in memory.
- **Errors are typed.** Raise the shared Error codes of Section 19.1 and attach the Langfuse trace_id. No silent `except` blocks.
- **Pinned and checked dependencies.** Exact versions; license reviewed (this matters for the reverse-engineering tool of Section 7); repository currency verified in the pre-step.
- **Format and lint.** ruff for style and banned-import rules; one formatter for the whole repository; CI fails on any warning.

21.4 Reference patterns

Model call through the gateway (R2, R5, R11). Budget and input checks come first, spend is recorded after, and output is treated as data.

```
async def call_model(agent_id: str, model: str, messages: list[Message]) -> Completion:
    verdict = guardrails.check_input(agent_id, messages)       # budget + input filter
    if not verdict.ok:
        raise ApiError(verdict.code, trace_id=current_trace()) # budget_exhausted, account_frozen,
                                                               # constitution_violation
    resp = await router.chat(model=model, agent_id=agent_id, messages=messages)
    await ledger.record_spend(agent_id, resp.usage)            # PostgreSQL, append-only
    return guardrails.check_output(resp)                       # output is data, not orders
```

Blue/Red loop with circuit breaker (R5, R8, Section 7). Limits come from config; the Arbiter joins from round 3.

```
async def blue_red_loop(subtask: Subtask, cfg: LoopLimits) -> LoopResult:
    history: list[Round] = []
    for rnd in range(1, cfg.max_rounds + 1):               # initial value: 5
        artifact = await blue.propose(subtask, history)
        run = await sandbox.run(artifact, ttl_seconds=cfg.sandbox_ttl)
        review = await red.attack(artifact, run)           # score in [0, 1] plus findings
        history.append(Round(rnd, artifact, run, review))
        if review.score >= cfg.accept_score:               # set from measurement
            return LoopResult.accepted(artifact, history)
        if rnd >= cfg.arbiter_from_round:                  # initial value: 3
            path = await arbiter.propose_common_path(history)
            subtask = subtask.with_guidance(path)
    return LoopResult.pruned(history)                      # Orchestrator switches GFlowNets path
```

Hybrid PRM score (R14, Section 8). Deterministic verifiers cap the model-based score; they are never overridden by it.

```
def prm_score(step: Step, checks: list[CheckResult], scorer_score: float, cfg: PrmCfg) -> Score:
    failed = [c for c in checks if not c.passed]           # compile, tests, schema, SymPy/SMT
    if failed:
        return Score(min(scorer_score, cfg.advance_threshold - 0.01),
                     evidence=[c.evidence for c in failed])      # cannot reach MCTS
    mode = "verified" if checks else "model_scored_only"         # weighted less by the Arbiter
    return Score(scorer_score, mode=mode)
```

Independence check on failover (R13, Section 13). A fallback that would put two roles on one family is skipped; if none is left, fail closed.

```
def overlaps(roles: dict[str, RoleModel], pairs: list[tuple[str, str]]) -> list[str]:
    return [f"{a}/{b}" for a, b in pairs if roles[a].family == roles[b].family]

def pick_fallback(role: str, candidates: list[RoleModel], roles, pairs) -> RoleModel:
    for cand in candidates:
        if not overlaps({**roles, role: cand}, pairs):
            return cand
    raise ApiError("circuit_open")                        # fail closed (R11)
```

Approval gate (R7). The executor checks the approvals table, never the agent's claim that approval was given.

```
PROTECTED = {"publish_shard", "merge_pr", "change_constitution", "expose_external_network",
             "ban_system", "freeze_budget", "activate_tool", "update_tool_version"}

def guard_action(action: str, evidence_ref: str) -> Response:
    if action in PROTECTED:
        req = approvals.create(action=action, evidence_ref=evidence_ref)
        return Response(202, {"code": "approval_required", "approval_id": str(req.id)})
    return Response(200)

def execute_protected(action: str, approval_id: UUID) -> None:
    if approvals.status(approval_id) != "approved":        # read from the database
        raise ApiError("approval_required")
    ...
```

Ledger entry (R9, Section 12). Each record carries the previous hash, so silent edits are detectable.

```
def append_entry(kind: str, payload: dict, prev: LedgerEntry) -> LedgerEntry:
    body = canonical_json({"kind": kind, "payload": payload, "prev_hash": prev.hash})
    h = sha256(body).hexdigest()
    return LedgerEntry(kind=kind, payload=payload, prev_hash=prev.hash,
                       hash=h, signature=ed25519_sign(SIGNING_KEY, h.encode()))
```

21.5 Rules for AI coding agents

Most of the code is written by AI coding agents (Section 17). These rules keep that safe and reviewable.

- **One module, one task.** Each task names its module, its interface (from Section 19), its limits and its "done" test. No task spans several layers.
- **Read first, write second.** The agent must read this section and the relevant schema before writing code, and must state which iron rules apply to the task.
- **Forbidden paths.** Agents may not edit `constitution/`, the signing keys, or CI configuration, and have no access to the acceptance set or the evaluator. A change that touches them is rejected automatically.
- **Tests come with the code.** A module without its tests is not a module. The agent that writes the code does not write the acceptance test that grades it (R8).
- **No guessing.** If an interface, threshold or library version is unknown, the agent writes a question into `docs/` rather than inventing a value (R12).
- **Small pull requests.** One module per PR, with a short description of what changed and which rules were checked. The agent opens a draft PR; a human merges (R7).

21.6 Testing rules

Layer	What it checks	Feeds metric (Section 15)
Unit	Pure logic: scoring, cell placement, coefficient × seconds settlement, hash chain	Accuracy
Property-based	Ledger tamper detection, budget never negative, liquidity share never above cap, a failed verifier always caps the PRM score	Economy, PRM calibration
Integration	Blue → sandbox → Red → Qdrant, with failover, independence checks and circuit breaker	Stability, Role independence
Red Team suites	stress, prompt_injection and constitution suites from `/redteam/tests`; sandbox escape attempts against each runtime	Security, Red Team effectiveness
Simulation	Reward/penalty coefficients, cartel behaviour, emergency protocols	Economy, Stability
Resource and memory	Full run in a container capped at 32 GB; mean and peak memory per service; MCTS-queue and Deep Sleep load together; measurement pass and enforcement pass	Resources, Peak memory, Time under limit
Soak	24-hour continuous run with crash and circuit-breaker counts	Stability
Acceptance (external)	Confidential acceptance set, scored by the external evaluator; canary scan for leaks	Accuracy, Test confidentiality, shard acceptance

- Every security rule has a test that tries to break it: a mutation that touches the constitution, a prompt hidden in web content, a write without a cross-query report, a fallback model from the same family.
- A flaky test is a defect, not noise. It is fixed or removed within the same PR.

21.7 Pull request and review flow

- AI agent opens a **draft PR** → CI runs lint, types, tests, secret scan, banned-call scan and the constitution hash check → Red Team suites run in the sandbox.
- A human reviews against the Definition of Done below and merges. Releases, shard publication and any constitution change use the approval flow of Section 19.8.
- Every merge records its trace_id and the measurements it affected in PostgreSQL, so results can be traced back to code.

21.8 Anti-patterns: never write these

- `eval`, `exec` or `subprocess` on text an agent produced, outside the sandbox (R3).
- Importing a model provider SDK inside agent code (R2).
- Pasting web or MCP content into a system prompt or treating it as a command (R4).
- Hard-coded thresholds, model names, keys or URLs (R5, R10, R12).
- A model approving, scoring or grading the artifact that same model produced, or any code path that reads, copies or logs acceptance-set items (R8).
- Letting a model-based score override a failed deterministic check (R14).
- Assuming a fallback model is independent without running the schema check (R13).
- Writing straight to Qdrant or the Cosmic Tree while skipping the cross-query gate (R6).
- Keeping state in container memory instead of PostgreSQL, Qdrant or Redis with a durable record; unbounded queues or batches.
- Catching every exception and continuing; unbounded retries or loops (R11).

- Launching a tool server per call instead of keeping a persistent one; calls without a deadline; two services writing the same entity (R15).
- Editing or deleting a ledger entry; “fixing” history instead of appending a correction (R9).

21.9 Definition of Done

✓	**Check**
☐	Module has an explicit interface that matches the schema in Section 19, and its tests pass.
☐	No iron rule (R1–R16) is broken; the PR names which rules it touched.
☐	All limits and thresholds come from `limits.yaml` and are labelled initial or measured.
☐	Red Team suites (stress, prompt_injection, constitution) pass with no critical finding.
☐	Failure paths are tested: timeout, budget exhausted, sandbox timeout, signature mismatch, no independent fallback.
☐	Logs carry a trace_id and contain no secrets; measurements are written with a date.
☐	Mean and peak memory are measured for modules that belong to the Phase 2 shard, and stay inside the limits of Section 5.
☐	A human has reviewed and merged the change.

These rules are proposals in the same sense as Section 19: they will be revised after the pre-step, but the iron rules R1–R16 are the baseline every revision must keep.

22. Low-Latency Integration and Synchronization

The system joins many components: Agno agents, the LiteLLM router, MCP tools (including REA), Qdrant, PostgreSQL, Redis/Celery, the sandbox, n8n, Langfuse and the dashboard. If each pair were wired point to point, every extra tool would add hops, serialization and new ways to fail. This section fixes one protocol per kind of interaction, keeps the hot path short and local, and makes state changes safe when several components act at once. The component that implements this map is the Bridge (Section 24). Like the rest of the paper it is a design: every latency figure below is a budget to be measured in the pre-step, not a result.

22.1 Design principles

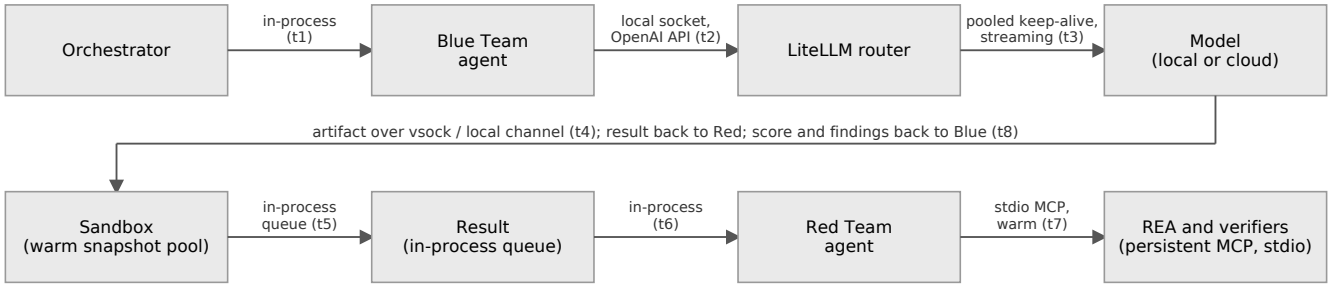
1. **One protocol per interaction type.** Agent-to-tool, agent-to-model, agent-to-agent, events and durable state each have one standard, so a new tool joins by speaking the existing contract instead of getting its own integration.
2. **Short, local hot path.** What happens inside one Blue/Red round stays on one host, on in-process calls, local sockets or virtio-vsock, and avoids public network hops.
3. **Async everywhere else.** Telemetry, indexing, dashboards and macro events leave the hot path through bounded queues and never block an agent.
4. **Every message has a deadline, a key and a trace.** A hop that cannot finish in the time left fails fast, a repeated message has no second effect, and one trace id follows the whole path.
5. **One owner per piece of state.** Components send commands to the owner; they never write the same entity directly.
6. **Measure first.** Published benchmarks disagree (see 22.8), so each budget is set from our own measurements.

22.2 Protocol map

Interaction	Standard and transport	Rule
Agent → tool	MCP (JSON-RPC). Tools inside the sandbox use stdio: the client launches the server as a subprocess, with no network hop. Shared tools use Streamable HTTP bound to localhost, with Origin validation.	Persistent server per sandbox, never launched per call. The MCP protocol revision is pinned in config, because revisions change session behaviour. Tool allowlist (Section 6).
Agent → model	OpenAI-compatible API through the LiteLLM router; local models over a local socket or loopback, cloud models over pooled keep-alive HTTPS.	The router is a measured hop: LiteLLM reports its own overhead in the `x-litellm-overhead-duration-ms` response header, which feeds the latency metric.
Agent ↔ agent, same shard	In-process calls when agents share a process; otherwise Redis Streams or pub/sub with the common envelope (19.14).	Keyed by entity_id for ordering; bounded length; consumer groups.
Agent ↔ agent, across shards (Phase 3)	A2A (Agent2Agent): Agent Cards for discovery, Tasks and Messages, JSON-RPC over HTTPS, gRPC or REST bindings, signed cards. The AEM acts as the gateway.	MCP stays agent-to-tool; A2A is only peer-to-peer between ecosystems. Authorization is ours to define; A2A defines identity, not permissions.
Events and telemetry	Redis pub/sub into PostgreSQL in batches; only macro events go to n8n (Section 5).	Never on the hot path.
Durable state	PostgreSQL (ledger, budgets, approvals, outbox) and Qdrant (vectors).	PostgreSQL is the source of truth; Qdrant is rebuilt from it if needed.
Host ↔ sandbox	Firecracker virtio-vsock from Phase 1; the local socket or mounted channel of the gVisor/Docker runtime in the pre-step.	No network inside the sandbox; the channel carries artifacts and results only.

22.3 Low-latency path

The figure shows one Blue/Red round. Each arrow is a hop with its transport and its own time budget (t1 to t8, set from measurement).



Techniques that shorten the path.

Technique	Where	Effect and cost
Warm sandbox pool from snapshots	Firecracker (Phase 1)	A cold microVM boot is on the order of 100 ms or more, while restoring a pre-booted snapshot is reported to be much faster (project reports vary and must be measured here). Each warm VM holds RAM, so the pool size is capped and counted in the peak-memory test.
Persistent connections and servers	Router, Qdrant, PostgreSQL, REA	Avoids per-call process start, TLS handshakes and tool-list reloads. REA is installed once, not launched with a package runner per call.
Streaming and early hand-off	Model → Blue → sandbox	Partial output flows to the next stage as soon as it is usable.

Technique	Where	Effect and cost
Parallel checks	PRM, Red Team	Deterministic verifiers run in parallel with the model-based scorer instead of after it (Section 8).
Hedged requests	Router	If a model exceeds its p95 for the call, a second family is asked and the first answer wins; independence rules of Section 13 still apply.
Batching	PRM scoring, Qdrant upserts, macro events	One call for many items; batch size bounded to protect memory.
Stable prompt prefixes	Constitution and system prompts	The unchanging beginning of every prompt lets servers that support prefix or prompt caching reuse work.
Small tool surface	MCP	Only the allowlisted tools are advertised, and REA's compact input-schema profile keeps the tool list small, which cuts tokens and time.
Deadline propagation	Every hop	Each hop subtracts its time from `deadline_ms`; a hop that cannot finish fails fast and the Orchestrator picks a cheaper path.
Backpressure	Queues	Full bounded queues make producers wait instead of allocating memory (Section 5).

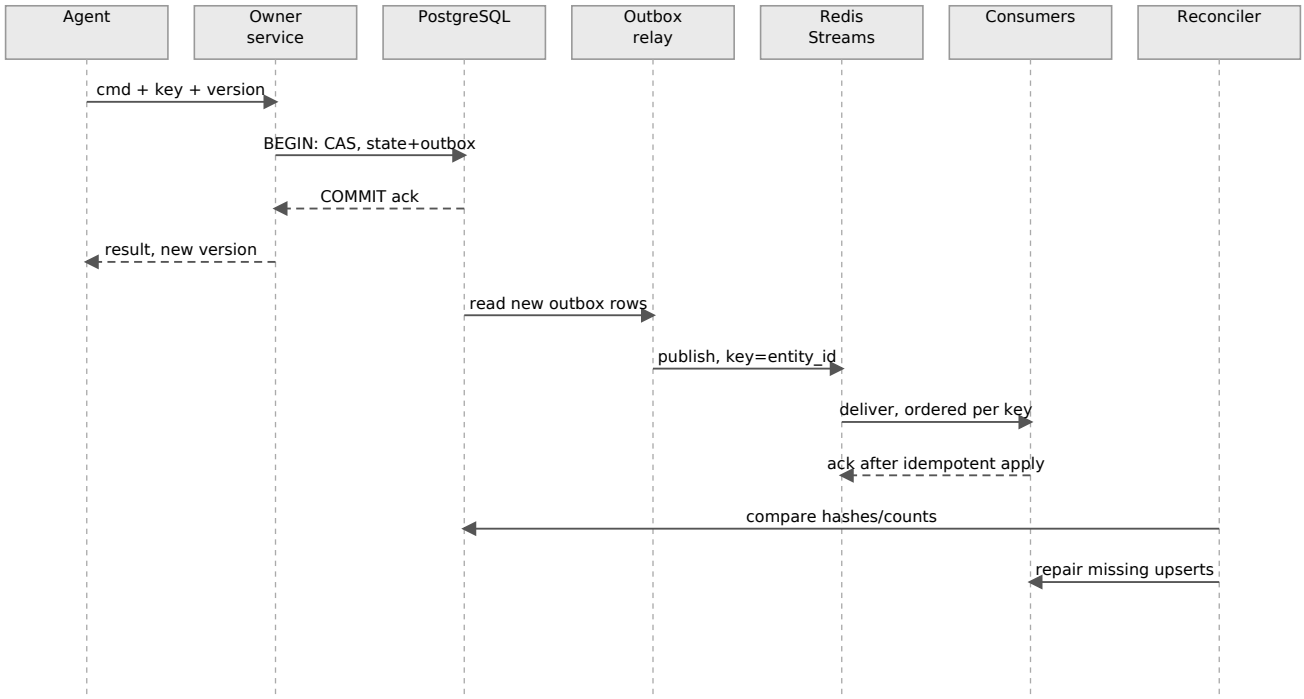
22.4 Latency budget

Each stage gets a budget. The values stay empty until the pre-step measures them; a hop that exceeds its budget in the 24-hour run is a defect.

Hop	Transport	Budget (p50 / p99)	How it is measured
t1 Orchestrator → Blue	in-process	Set by measurement	Trace span
t2 Agent → router	local socket	Set by measurement	Trace span
t3 Router → model (incl. router overhead)	keep-alive HTTPS or local	Set by measurement	Router overhead header, trace span
t4 Blue → sandbox acquire and run	vsock, warm pool	Set by measurement	Sandbox runner timings
t5–t6 Result → Red	in-process queue	Set by measurement	Trace span
t7 Red → REA / verifier	stdio MCP	Set by measurement	MCP call timings
t8 Score back to Blue	in-process	Set by measurement	Trace span
End to end per round	all of the above	Set by measurement	Sum of spans, with queueing time shown separately

22.5 Synchronization path

Speed is useless if components disagree. The design below keeps one source of truth, one writer per entity and safe retries, and lets slower consumers catch up without blocking agents.



- **Single writer per entity.** Each agent account, archive cell, task and ledger has one owning service. Others send commands. The hash-chained ledger is sequential by nature, so it has exactly one writer.
- **Compare-and-set.** Writes carry `expected_version`. This closes the MAP-Elites race (“replace only if better”): two agents cannot both replace the same cell from a stale read, because the second compare fails and retries.
- **Transactional outbox.** The state change and its event are written in one PostgreSQL transaction, and a relay publishes the outbox rows. No event is lost if a crash falls between the database and the queue.
- **At-least-once plus idempotence.** Streams may deliver twice, so consumers apply each `idempotency_key` once and record their offset.
- **Ordering only where needed.** Messages are ordered per `entity_id` (the stream key). No global order is required, which keeps the hot path parallel.
- **Leases and fencing.** Work items (such as the 2-minute Red Team request) are leases with a monotonically increasing fencing token; a stale worker’s late write is rejected.
- **Consistency tiers.** Strong: ledger, budgets, approvals, constitution (PostgreSQL). Eventual: Qdrant indexing, dashboard, n8n. Cache: Redis, with a time-to-live. A Qdrant write may lag, but it is never the only record.
- **Reconciliation.** During Deep Sleep a reconciler compares PostgreSQL and Qdrant counts and hashes and repairs gaps.
- **Time.** Deadlines use monotonic clocks on one host; across hosts, ordering uses trace ids and sequence numbers, never wall-clock comparison.
- **Across shards.** The AEM needs no cross-shard transaction: shards send signed, idempotent, append-only reports, and the AEM audits them eventually (Section 13).

22.6 Message bus choice

Redis is already in the stack, so Redis Streams (append-only log, consumer groups, bounded length) adds no new service to the 32 GB budget, and is the default. NATS (with JetStream for persistence) is the alternative if a multi-node Phase 1 shows that Redis becomes the bottleneck or the memory cost; its request-reply and subject routing suit service-to-service calls. The choice is made from measurement, not from published numbers, and the envelope of Section 19.14 means switching the bus does not change agent code.

22.7 Failure and degradation

- A hop that misses its deadline returns `sandbox_timeout`, `circuit_open` or `budget_exhausted` from the shared error set and does not retry blindly.
- Optional steps (extra tools, hedged requests, GFlowNets width) are dropped before required ones when time is short.
- A slow or full consumer cannot slow an agent: queues are bounded and the producer sees backpressure.
- Every new component or tool must declare its protocol, its transport, its deadline and its owner before it joins; otherwise it is not added.

22.8 References

Sources used for this section, to be re-checked in the pre-step because these projects change quickly:

- Model Context Protocol, transports (stdio and Streamable HTTP): `modelcontextprotocol.io/specification`.
- Agent2Agent (A2A) protocol, governed under the Linux Foundation: `a2a-protocol.org`.
- LiteLLM gateway benchmarks and the overhead header: `docs.litellm.ai/docs/benchmarks`. A competing gateway vendor publishes a very different overhead for LiteLLM, which is why this paper measures its own.
- Firecracker snapshot restore and warm pools: public project reports (for example `fly.io/learn/firecracker-vm`); figures differ between projects and hardware.
- NATS and Redis Streams: vendor and community documentation and blog benchmarks, used only as a direction, not as figures.
- REA: `github.com/morluto/rea` and `rea.tools`.

23. Business Model and Economic Feasibility

This section asks whether the project can pay for itself, and how to find out cheaply. It is a proposal. Nothing here is implemented, no real money moves in the current scope (Section 12), and every payment, royalty and fee idea waits for a legal review (Section 14). The paper still gives no invented prices or revenue: it defines the cost model, the stages and the conditions, and the pre-step supplies the numbers.

23.1 Value and customers

- **What is unique.** Verifiable AI-written code: separate builder and attacker, signed evidence, a tamper-evident ledger, human approval, and a design that runs locally in 32 GB.
- **Individual developers and small teams** who want a local, private agent system and do not want to send source code to a cloud service.
- **Security-conscious companies** that need an audit trail for AI-generated code (who proposed, who attacked, what evidence, who approved).

- **Researchers and educators** who need a reproducible multi-agent evaluation harness.
- **AI teams** that need independent evaluation of agent output (the external evaluator and Red Team as a service).

23.2 Revenue streams

Stream	What is sold	Who pays	Earliest phase	Main risk
Verification and audit service	Signed evidence reports: Red Team results and external-evaluator scores on a customer's AI-generated code	Companies, AI teams	Pre-step to Phase 1 (pilots)	Needs credible evidence quality; liability for a missed bug
Open-core	Free shard; paid modules: governance dashboard, audit export, SSO, managed AEM audit	Companies	Phase 2–3	A free version that is good enough; support load
Managed cloud shard	Hosted shard for people without a 32 GB machine; compute cost plus margin	Individuals, teams	Phase 2–3	Margin depends on model cost and mix of local and cloud models
Compute rental	Shards rent spare compute to each other, with a very small fee per transaction	Shard owners	Phase 3	Payments regulation; fraud and abuse
Knowledge and royalty	Verified Cosmic Tree leaves and evolution objects; creator share (for example 10%)	Shard owners	Phase 3–4	IP ownership and quality; payments regulation
Support, training, sponsorship, grants	Help, courses, open-source sponsorship, research grants, cloud credits	Users, sponsors, institutions	Any	Small and uneven income

The verification service is the likeliest first revenue: it needs no marketplace, no payments between users and little capital, and it uses what the pre-step builds anyway. The marketplace streams carry the most regulatory risk and come last.

23.3 Cost model and unit economics

Revenue only matters relative to the cost of one accepted task. The cost drivers follow directly from the architecture: five reasoning paths (GFlowNets), up to five Blue/Red rounds, a second model for the Red Team, a sandbox run per round and a human approval for protected actions. With T the tokens per call, p the token price of the model used, R the mean rounds per task, n the number of model calls per round, and A the share of tasks that end accepted:

```
cost per attempt   C_attempt = R * ( n * T * p_mix  +  c_sandbox )  +  c_prm  +  c_review
cost per accepted  C_task    = C_attempt / A
gross margin       M         = (P - C_task) / P              P = price per accepted task
break-even volume  N_BE      = F / (P - C_task)              F = fixed cost per month
```

Here *p_mix* blends local and cloud model prices (local models cost electricity and hardware wear, cloud models cost per token), *c_prm* covers the PRM and verifier work, *c_review* is the human approval time, and *F* covers the Phase 1 server, the external-evaluator machine, tools and any staff. All are measured in the pre-step (Section 15, "Unit cost").

Driver	If it rises	Lever
Rounds R	Cost rises roughly in proportion	Prune dead branches early (PRM threshold), better Blue first drafts, cap rounds

Driver	If it rises	Lever
Cloud share in p_mix	Cost per token rises, local cost falls	Blue on a local model, cloud only for Red and Consensus where independence needs it (Section 13)
Acceptance A	Cost per accepted task falls	Better calibration, reuse of Cosmic Tree leaves, easier task selection
Review time c_review	Cost and latency rise	Approve only protected actions (Section 11); batch approvals
Fixed cost F	Break-even volume rises	Self-hosted shards put hardware cost on users; credits and grants cover Phase 1

The economy of Section 12 is a model of this cost. The internal token is only an accounting unit, but its budgets can be denominated against real cost per call, so the simulation predicts real spend. Converting internal tokens into money is a separate step that needs the legal review.

Why the shard changes the economics. A self-hosted shard runs on the user's hardware, so the variable cost of serving that user is close to zero, and paid services sit on top as options. This matters for a project with very limited funds: capital is needed for the Phase 1 server and the evaluator, not for serving every user.

23.4 Feasibility stages and gates

1. **Pre-step: measure.** Record tokens, rounds, sandbox seconds, review minutes and acceptance rate per task. Fill the cost model. *Gate:* a measured C_task exists for at least one task type.
2. **Pilots with design partners.** Offer the verification service to a few teams, free or at cost, and learn what they would pay for. *Gate:* at least one customer segment where price clearly exceeds measured C_task.
3. **Open-source shard (Phase 2).** Publish, build community and a contributor base. *Gate:* the Phase 2 acceptance test, plus a license choice for the code.
4. **Paid tiers.** Open-core modules and managed shard, after the legal review of terms and payments.
5. **Network streams (Phase 3–4).** Compute rental and royalties only once the AEM, the audit and the legal review exist.

Stop conditions. If the measured cost of an accepted task cannot be brought below what any segment will pay, the project continues as an open research and open-source effort and does not take outside money on promises it cannot meet.

23.5 Funding path

- **Non-dilutive first:** cloud credits, startup and education programs, research grants, hackathons, open-source sponsorship funds and university or community partnerships. These match the Phase 1 needs of Section 17.
- **Earned revenue next:** pilot fees for the verification service and support.
- **Only later, if ever:** equity or investment, after the cost model and pilots show a viable segment.
- **Legal capacity.** The author is a minor, so contracts, payments, accounts and company formation need a legal guardian and professional legal and tax advice under local law.

23.6 Competition and defensibility

Many agent frameworks exist, and large AI labs can add review and sandboxing to their own products. The project's claim to a place is narrower: independent verification with signed evidence, governance by code and humans, and a local 32 GB footprint. These are hypotheses until measured. Possible lasting advantages are the evidence format, an audited archive of verified solutions, and a community around the shard; none is proven yet.

23.7 Legal, ethical and licensing notes

- Royalties, fees and compute rental are financial activities that may be regulated; nothing is enabled before legal review (Sections 12, 14).
- AI-written code raises liability and intellectual-property questions; the evidence trail is meant to help, not to guarantee.
- Security tooling is dual-use. Red Team and REA capabilities are used within lawful scope, on artifacts we produce or are authorized to examine (Section 7).
- Third-party licenses are respected and not hidden: REA is MIT-licensed, and commercial analysis engines are never bundled. The license of this project's own code is an open decision for the Phase 2 gate.
- Local-first design is a privacy advantage: customer code stays on the customer's machine unless they choose a cloud model.

23.8 What will be measured

The unit-cost metric joins Section 15, and these business indicators are added once pilots start: cost per accepted task, price and margin per segment, pilot-to-paid conversion, shard installs (the Phase 3 trigger of 100), and support hours per user. Each is written to PostgreSQL with a date, like every other measurement.

24. The Bridge: Unified Tool Integration Layer

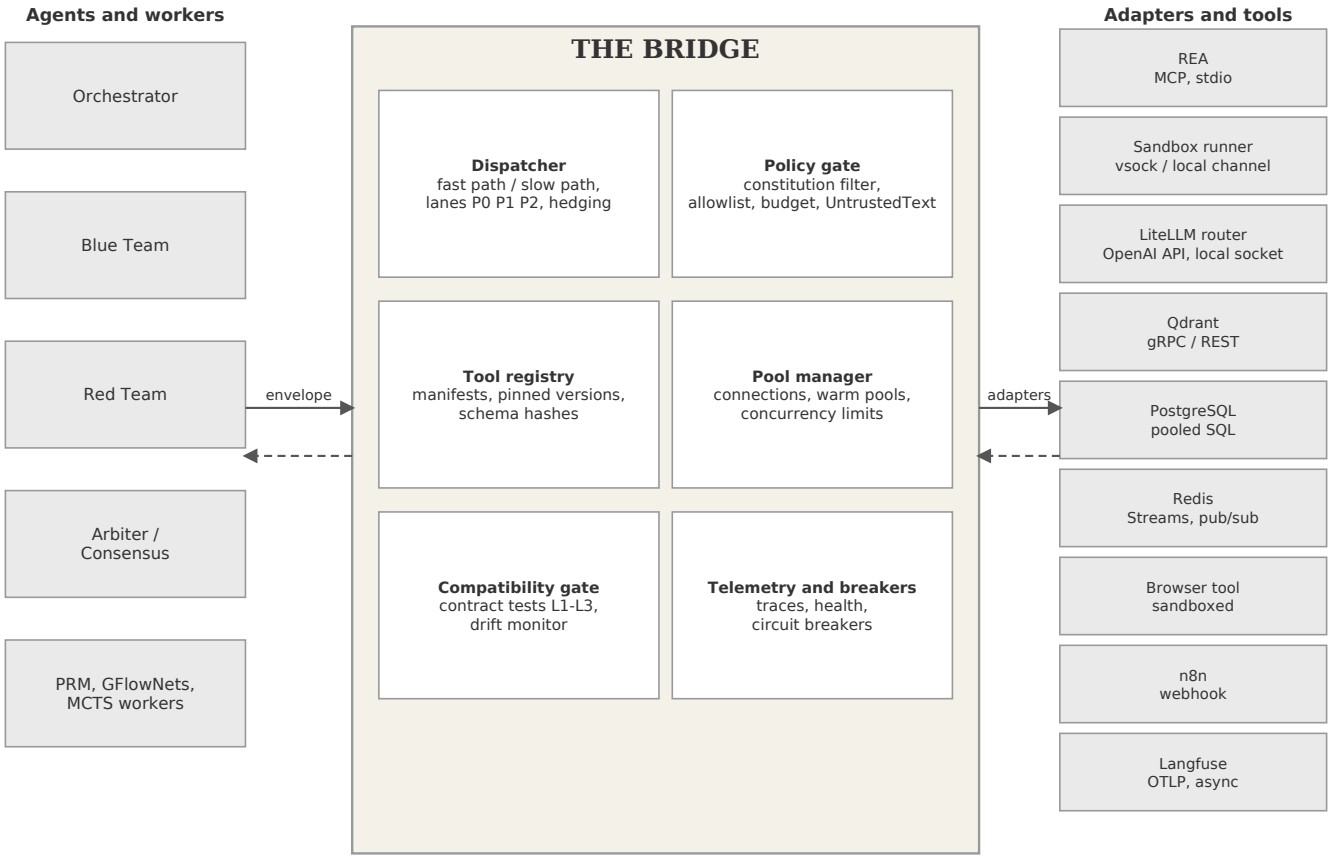
Section 22 sets the principles and the protocol map. This section defines the component that carries them out: the **Bridge** (in Turkish, *köprü*), the single layer through which every agent reaches every tool. Four properties are its job: **full compatibility** (any tool fits or is not admitted), **high throughput** (many calls at once without choking), **low latency** and **instant responsiveness** (the first useful result arrives early, not only the last). Like the rest of the paper, this is a design: every number is a budget to be measured.

24.1 Goals, mechanisms and measures

Goal	Mechanism	Measured by
Full compatibility	One canonical contract (envelope plus MCP-style tool calls), tool manifests, pinned versions and schema hashes, contract tests at three levels, drift monitor, error normalization	100% of active tools pass L1–L3; every drift caught before use
High throughput	Priority lanes, credit-based admission, per-tool limiters, batching and pipelining, connection pools, artifact references instead of payloads	Sustained calls per second per tool within its p99 budget
Low latency	Fast path on one host (in-process, local socket, vsock, stdio), persistent servers, warm pools, small payloads, no logging on the hot path	p50 and p99 per hop; bridge overhead per call
Instant responsiveness	Streaming partial results, prefetch, speculative side-effect-free work, early cancellation, hedging	Time to first partial result, separate from completion time

24.2 Architecture

Agents never hold a tool client. They send an envelope to the Bridge, which checks the policy, finds the tool in its registry, takes a pooled connection and calls a thin adapter. Adapters translate between the canonical call and each tool's native protocol and keep no state, so they can be restarted or replaced freely.

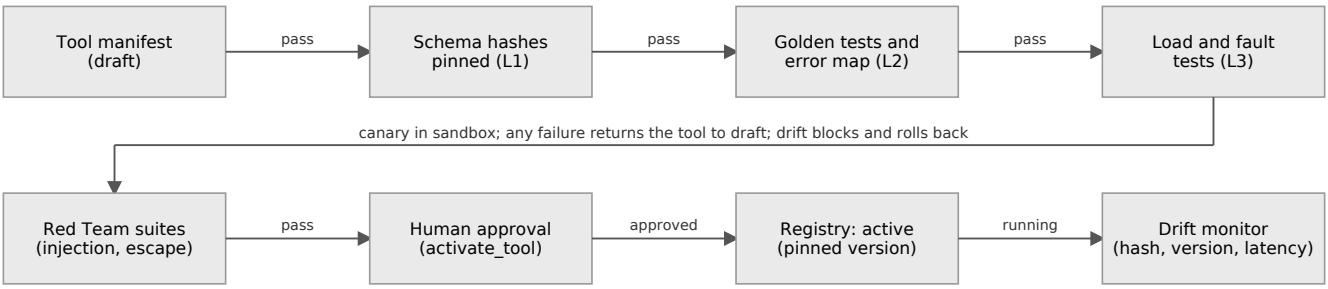


- **Dispatcher.** Chooses the path (see 24.5), the lane and whether a hedge is allowed, and enforces the deadline.
- **Policy gate.** Runs the constitutional filter, the MCP and tool allowlist and the budget check, and wraps untrusted output (web, REA) as `UntrustedText`. It is code, not an agent, and it cannot touch the constitution.
- **Tool registry.** Holds one manifest per tool: exact version, integrity hash, protocol, transport, owner, per-method deadlines, side effects, idempotency, capabilities, limits and state (draft, canary, active, degraded, retired). Schema in Section 19.16.
- **Pool manager.** Keeps persistent connections and warm instances (REA server, sandbox snapshots, model connections), with limits that count toward the peak-memory budget.
- **Compatibility gate.** Runs contract tests and watches for drift (24.3).
- **Telemetry and breakers.** Emits spans asynchronously, tracks health per tool and opens a circuit breaker when a tool misbehaves.

24.3 Full compatibility

Compatibility is enforced by gates, not hoped for. A tool is admitted in four levels of fit, and only the last makes it active:

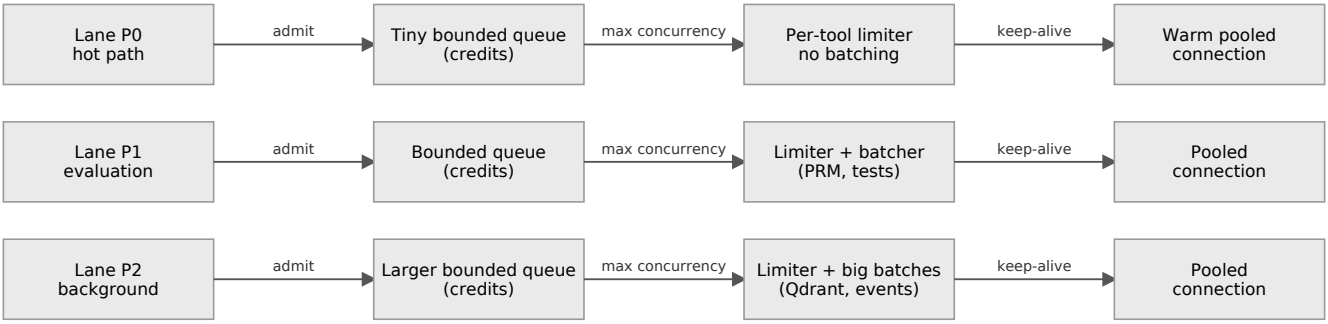
1. **L0 reachable.** The adapter can start or connect to the tool inside the sandbox canary.
2. **L1 schema match.** Input and output schemas match the manifest, and their hashes are pinned. Any later change is `schema_drift` and blocks calls.
3. **L2 behaviour match.** Golden requests and responses pass, including error cases: every native error maps to the shared error set (timeouts, rate limits, invalid input, unavailable), and idempotent methods really are idempotent.
4. **L3 operational fit.** Latency, throughput and memory stay inside the tool’s budget under load and under injected faults (slow, killed, malformed output). Only then can the tool become active, and activation is a protected action that needs human approval.



- **Canonical data rules.** UTF-8 text, UUID identifiers, UTC RFC 3339 timestamps, size limits per method, and large artifacts passed by reference (`artifact_ref`) rather than by value.
- **Capability negotiation.** A manifest declares streaming, batching, cancellation and idempotency. The Bridge uses what exists and degrades gracefully when something is missing, for example by splitting a batch or polling instead of streaming.
- **Versions.** An exact version is pinned; a compatible range is watched. The MCP protocol revision is pinned too. An update is a pull request that reruns L1–L3 and the Red Team suites (Section 7, supply chain).
- **One owner per tool state.** Each tool's registry entry has a single owning service, as in Section 22.5.

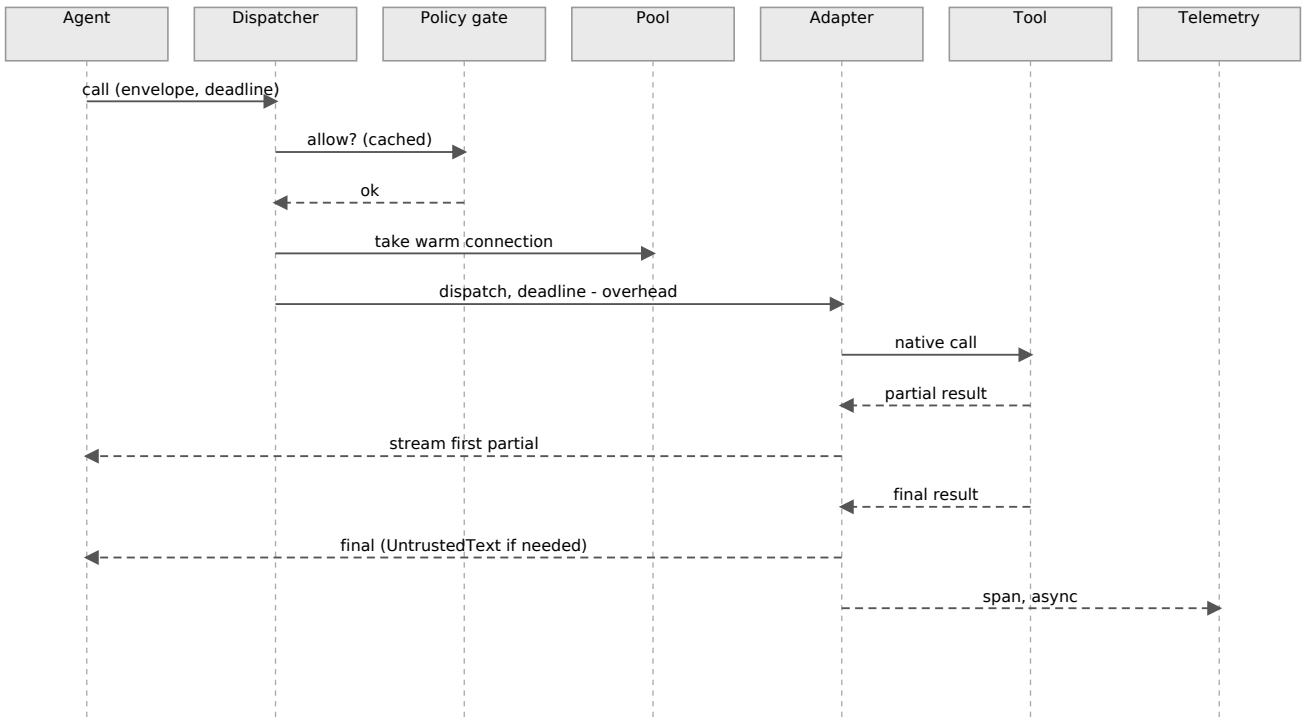
24.4 High throughput

The Bridge protects the hot path by sorting work into lanes. Each lane has a bounded queue and credits; a producer without credit waits (backpressure), so a flood of background work cannot use the memory or the connections that a Blue/Red round needs.



- **Per-tool limiters.** A maximum concurrency per tool, set in its manifest, prevents one tool from being overrun, and keeps the 32 GB budget.
- **Batching where it is safe.** Qdrant upserts, PRM scoring and macro events travel in batches. The hash-chained ledger stays sequential, but several entries can share one transaction.
- **Pipelining.** Requests to a tool that supports it are sent before earlier answers return: JSON-RPC batches where the transport allows, multiplexed streams for HTTP.
- **Pools sized from measurement.** Pool sizes follow the measured core count and memory, not a guess.
- **Caches.** Read-through caches with a time-to-live for policy decisions, tool schemas and Cosmic Tree retrievals; a cache never serves a protected or constitutional value.
- **References, not payloads.** Binaries, logs and archives are shared by reference on disk, so the Bridge moves small messages and not large blobs.

24.5 Low latency and instant responsiveness



- **Fast path.** When the tool is on the same host, the Bridge calls it in-process, or over a local socket, stdio or vsock, without leaving the machine and without re-serializing objects it can pass directly. The slow path (the message bus) is used for work that may wait.
- **Deadline accounting.** The Bridge subtracts its own overhead and the pool wait from `deadline_ms` before it calls the tool, and refuses a call that cannot finish in time (`deadline_exceeded`).
- **Warm everything.** Persistent REA server, warm sandbox snapshot pool, open model connections, prepared database statements.
- **Prefetch and speculation.** While the Blue Team starts a draft, the Bridge can prefetch likely Cosmic Tree retrievals; the Red Team can start static checks on partial output. Only side-effect-free methods may run speculatively, and a result that the draft invalidates is discarded.
- **Streaming first.** Partial results flow to the caller as soon as they are usable, so time to first partial result stays small even when completion takes longer.
- **Cancellation.** When a branch is pruned or a deadline passes, a cancel is forwarded to the tool so it stops using CPU and memory.
- **Hedging.** For side-effect-free calls that pass their p95, a second request goes to another instance or model family; the first answer wins and the other is cancelled. Independence rules of Section 13 still apply.
- **Tail latency.** The hot path runs in processes with priority and memory limits, logging is asynchronous, and the slowest percentile (p99) is tracked per tool because it, not the average, decides how fast a round feels.

24.6 Tools the Bridge connects

Tool	Role	Native protocol	Transport on the fast path	Notes
Agno agents, TorchGFN, DSPy	Agents, reasoning, offline prompt optimization	Python objects	In-process	No adapter needed; still counted by the policy gate

Tool	Role	Native protocol	Transport on the fast path	Notes
LiteLLM router	Model access, failover	OpenAI-compatible API	Local socket or loopback; pooled HTTPS to cloud	Reports its own overhead in a response header
REA	Reverse engineering	MCP	Persistent stdio server in the sandbox	Pinned version; output untrusted; static inspectors first
Sandbox runner	Run agent code	Runtime API	vsock (Firecracker) or local channel (gVisor/Docker)	Warm snapshot pool; no network
Browser tool	Fresh web data	Library or MCP	Inside the sandbox	Writes to the web pool as data only
Qdrant	Cosmic Tree, archive	gRPC or REST	Loopback	Batched upserts; disk-backed
PostgreSQL	Ledger, budgets, outbox	SQL	Pooled local connections	Source of truth; single writer per entity
Redis / Celery	Streams, queue	RESP	Local socket	Bounded streams; memory-limited
n8n	External automation	Webhook	HTTP, batched	Macro events only
Langfuse and dashboard	Traces, read-only views	OTLP, SQL	Async, off the hot path	Never blocks a call
Other shards (Phase 3)	Inter-ecosystem work	A2A	HTTPS or gRPC through the AEM	Signed Agent Cards

24.7 Safety inside the Bridge

- Every call passes the policy gate; nothing reaches a tool without the allowlist, the constitutional filter and the budget check, and every call is logged with its trace id.
- The Bridge is code, not an agent. It has no write access to the constitution, the signing keys or the acceptance set.
- Tools that run code or touch the outside world run only inside the sandbox; REA and web results are always `UntrustedText`.
- Activating a tool or changing its pinned version is a protected action (`activate_tool`, `update_tool_version`) that needs human approval.
- The kill switch closes all Bridge routes. If the Bridge itself fails, it fails closed (rule R11): calls are refused, not sent unchecked.

24.8 Deployment shapes and budget

Stage	Shape	Notes
Pre-step and shard (32 GB)	Bridge as an in-process library plus local sockets; one process group	Registry and pools are small and capped; counted in the peak-memory test
Phase 1 server	Bridge as a separate service with several workers; optional NATS if measurement asks	Per-tool pools can be larger; evaluator stays outside the agent environment
Phase 3 network	An A2A gateway at the AEM, with signed Agent Cards	The Bridge stays local to each shard; cross-shard calls are asynchronous and idempotent

24.9 Degraded modes

Failure	Bridge behaviour
A tool is slow	Deadline-aware shedding; hedge if side-effect-free; open the circuit breaker if it persists
A tool is down	`tool_unavailable`; optional steps are skipped, required steps wait or the branch is pruned
Schema drift detected	Block calls to that tool, mark it degraded, roll back to the pinned version, ask for approval
A lane is full	`lane_full` (backpressure); P2 work waits, P0 keeps its credits
Bridge overhead rises	Alert; the bridge-overhead metric (Section 15) fails the gate until fixed
Memory near the limit	Pools shrink, warm instances are released, P2 pauses (Section 5)

24.10 Acceptance

The Bridge passes its gate when, over the 24-hour run of Section 15, every active tool has a passing L1–L3 report and a pinned version, no drift reached a call, every native error was normalized, the Bridge's own overhead and memory stay inside their measured budgets, and the p99 of each hop stays inside its budget (Section 22.4). The thresholds are set from the pre-step measurements.